

# Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

## 1. Data Preparation

### 1.1. Loading the dataset

#### 1.1.1. Sample the data and combine the files

```
# Iterating through the monthly data:
# read a month file -> day -> hour: append sampled data -> move to next hour -> move to next day after 24 hours -> move to next month file
# Create a single dataframe for the year combining all the monthly data

# Select the folder having data files
import os

# Select the folder having data files
os.chdir('/content/data/')

# Create a list of all the twelve files to read
file_list = os.listdir()

# initialise an empty dataframe
df = pd.DataFrame()
print(file_list)

# iterate through the list of files and sample one by one:
for file_name in file_list:
    try:
        # file path for the current file
        file_path = os.path.join(os.getcwd(), file_name)

        # Reading the current file
        df_month = pd.read_parquet(file_path)

        # We will store the sampled data for the current date in this df by appending the sampled data from each hour to this
        # After completing iteration through each date, we will append this data to the final dataframe.
        sampled_data = pd.DataFrame()

        # Loop through dates and then loop through every hour of each date
        for date in df_month['tpep_pickup_datetime'].dt.date.unique():
            for hour in range(24):
                hour_data = df_month[(df_month['tpep_pickup_datetime'].dt.hour == hour) \
                                     & (df_month['tpep_pickup_datetime'].dt.date == date)]
                sample = hour_data.sample(frac = 0.05, random_state = 42)
                sampled_data = pd.concat([sampled_data, sample])
```

```
# Iterate through each hour of the selected date

# Sample 5% of the hourly data randomly

# add data of this hour to the dataframe

# Concatenate the sampled data of all the dates to a single dataframe
df = pd.concat([df,sampled_data])# we initialised this empty DF earlier

except Exception as e:
    print(f"Error reading file {file_name}: {e}")
```

After combining the data files into one DataFrame, convert the new DataFrame to a CSV or parquet file and store it to use directly. Ideally, you can try keeping the total entries to around 250,000 to 300,000.

```
# Store the df in csv/parquet
df.to_parquet('combined.parquet')
```

Iterating through all the files and taking date and hour data and then sampling 5% from the hour\_data and concatenating it with the df. Finally, writing the combined df to a parquet file called '**combined.parquet**'

## 2. Data Cleaning

### 2.1. Fixing Columns

#### 2.1.1. Fix the index

```
[18]
✓ Os  # Fix the index and drop any columns that are not needed
df.reset_index(drop=True, inplace=True)
df.drop('store_and_fwd_flag', axis = 1,inplace=True)
```

```
[19]
✓ Os  df.info()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 1889084 entries, 0 to 1889083
Data columns (total 18 columns):
#   Column                                Dtype
---  -
0   VendorID                             int64
1   tpep_pickup_datetime                 datetime64[us]
2   tpep_dropoff_datetime                 datetime64[us]
3   passenger_count                       float64
4   trip_distance                         float64
5   RatecodeID                           float64
6   PULocationID                         int64
7   DOLocationID                         int64
8   payment_type                          int64
9   fare_amount                          float64
10  extra                                float64
11  mta_tax                               float64
12  tip_amount                           float64
13  tolls_amount                          float64
14  improvement_surcharge                 float64
15  total_amount                          float64
16  congestion_surcharge                  float64
17  airport_fee                           float64
dtypes: datetime64[us](2), float64(12), int64(4)
memory usage: 259.4 MB
```

Since the combined file has improper index, I have reset it and dropped the index column and also 'store\_and\_fwd\_flag' column is not need for this task. Hence, I removed it.

### 2.1.2. Combine the two airport\_fee columns

```
 # Combine the two airport fee columns

df['airport_fee'].fillna(0, inplace=True)
df['Airport_fee'].fillna(0, inplace=True)
df['airport_fee'] = df['airport_fee'] + df['Airport_fee']
df.drop(['Airport_fee'], axis=1, inplace=True)
```

Filling null values with 0 in both airport\_fee column and then adding those columns and dropping Airport\_fee column. Now we have only one airport\_fee column in our dataframe.

### 2.1.3 [5 marks]

Fix columns with negative (monetary) values

[10]  
✓ Os

```
# check where values of fare amount are negative
df[df['fare_amount'] <= 0]
```

▼

| ...     | VendorID | tpep_pickup_datetime | tpep_dropoff_datetime | passenger_count | trip_distance | RatecodeID | store |
|---------|----------|----------------------|-----------------------|-----------------|---------------|------------|-------|
| 3119    | 1        | 2023-01-01 19:16:54  | 2023-01-01 19:17:15   | 1.0             | 0.00          | 2.0        |       |
| 3966    | 2        | 2023-01-02 05:12:19  | 2023-01-02 05:41:45   | 1.0             | 17.07         | 3.0        |       |
| 5039    | 1        | 2023-01-02 13:44:07  | 2023-01-02 13:48:36   | 0.0             | 0.00          | 1.0        |       |
| 7701    | 2        | 2023-01-03 08:27:38  | 2023-01-03 08:59:16   | 1.0             | 8.34          | 1.0        |       |
| 9093    | 2        | 2023-01-03 14:24:45  | 2023-01-03 14:25:14   | 1.0             | 0.00          | 2.0        |       |
| ...     | ...      | ...                  | ...                   | ...             | ...           | ...        | ...   |
| 1866611 | 1        | 2023-12-27 20:51:43  | 2023-12-27 21:00:02   | 3.0             | 0.00          | 5.0        |       |
| 1867582 | 1        | 2023-12-24 02:28:52  | 2023-12-24 02:36:27   | 0.0             | 1.40          | 5.0        |       |
| 1879409 | 1        | 2023-12-29 16:02:56  | 2023-12-29 16:03:31   | 0.0             | 0.00          | 5.0        |       |
| 1879545 | 2        | 2023-12-29 17:32:20  | 2023-12-29 17:33:49   | 1.0             | 0.02          | 1.0        |       |
| 1886132 | 1        | 2023-12-31 10:32:20  | 2023-12-31 10:32:32   | 1.0             | 0.00          | 5.0        |       |

649 rows x 19 columns

There are 649 rows with fare\_amount as 0.

Did you notice something different in the RatecodeID column for above records?

```
# Analyse RatecodeID for the negative fare amounts
df[df['fare_amount'] <= 0][['RatecodeID']].value_counts()
```

| RatecodeID | count |
|------------|-------|
| 1.0        | 378   |
| 5.0        | 158   |
| 2.0        | 56    |
| 99.0       | 24    |
| 4.0        | 13    |
| 3.0        | 4     |

dtype: int64

99.0 is not a valid RatecodeID and we have more rows with RateCodeID as 1.

```
df.loc[df['RatecodeID'] == 99.0, 'RatecodeID'] = df['RatecodeID'].mode()[0]
```

```
df['RatecodeID'].value_counts()
```

```
...
count
RatecodeID
1.0    1733019
2.0     71432
5.0     10248
3.0      6102
4.0      3716
6.0         3
```

```
dtype: int64
```

RateCodeID – 99.0 is invalid as per our dataset. So I am replacing it with mode value. Now we have only valid RateCodeID in a column.

## Find which columns have negative values

```
# Find which columns have negative values
cols_to_check = ['passenger_count', 'trip_distance', 'RatecodeID',
                  'PULocationID', 'DOLocationID', 'payment_type', 'fare_amount', 'extra',
                  'mta_tax', 'tip_amount', 'tolls_amount', 'improvement_surcharge',
                  'total_amount', 'congestion_surcharge', 'airport_fee']

min_values_series = df[cols_to_check].min()

# Filter this series to find columns where the minimum value is less than 0
negative_min_columns = min_values_series[min_values_series < 0]

print("Columns with negative values and their minimums:")
print(negative_min_columns)
```

```
Columns with negative values and their minimums:
extra                -2.50
mta_tax              -0.50
improvement_surcharge -1.00
total_amount        -5.75
congestion_surcharge -2.50
airport_fee         -1.75
dtype: float64
```

Columns – extra, mta\_tax, improvement\_surcharge, total\_amount, congestion\_surcharge, Airport\_fee are having negative values.

Fixing those negative values by using their median value.

```
# Fix these negative values by replacing them with the median of each column

# Calculate medians for each column with negative values
median_extra = df['extra'].median()
median_mta_tax = df['mta_tax'].median()
median_improvement_surcharge = df['improvement_surcharge'].median()
median_total_amount = df['total_amount'].median()
median_congestion_surcharge = df['congestion_surcharge'].median()
median_airport_fee = df['airport_fee'].median()

# Apply corrections using .loc to avoid SettingWithCopyWarning
df.loc[df['extra'] < 0, 'extra'] = median_extra
df.loc[df['mta_tax'] < 0, 'mta_tax'] = median_mta_tax
df.loc[df['improvement_surcharge'] < 0, 'improvement_surcharge'] = median_improvement_surcharge
df.loc[df['total_amount'] < 0, 'total_amount'] = median_total_amount
df.loc[df['congestion_surcharge'] < 0, 'congestion_surcharge'] = median_congestion_surcharge
df.loc[df['airport_fee'] < 0, 'airport_fee'] = median_airport_fee
```

## 2.2. Handling Missing Values

### 2.2.1. Find the proportion of missing values in each column

```
# Find the proportion of missing values in each column  
df.isnull().mean()
```

```
...  
0  
VendorID 0.000000  
tpep_pickup_datetime 0.000000  
tpep_dropoff_datetime 0.000000  
passenger_count 0.034177  
trip_distance 0.000000  
RatecodeID 0.034177  
PULocationID 0.000000  
DOLocationID 0.000000  
payment_type 0.000000  
fare_amount 0.000000  
extra 0.000000  
mta_tax 0.000000  
tip_amount 0.000000  
tolls_amount 0.000000  
improvement_surcharge 0.000000  
total_amount 0.000005  
congestion_surcharge 0.034177  
airport_fee 0.000000
```

```
dtype: float64
```

We have null values in Passenger\_count, RateCodeID, total\_amount and congestion\_surcharge columns.

### 2.2.2. Handling missing values in passenger\_count

```
# Display the rows with null values
# Impute NaN values in 'passenger_count'
df[df['passenger_count'].isnull()]
```

|         | VendorID | tpep_pickup_datetime | tpep_dropoff_datetime | passenger_count | trip_distance | RatecodeID | PULocationID |
|---------|----------|----------------------|-----------------------|-----------------|---------------|------------|--------------|
| 4       | 2        | 2023-01-01 00:43:00  | 2023-01-01 01:01:00   | NaN             | 19.24         | NaN        | 66           |
| 15      | 2        | 2023-01-01 00:41:50  | 2023-01-01 01:14:50   | NaN             | 10.77         | NaN        | 151          |
| 42      | 2        | 2023-01-01 00:37:21  | 2023-01-01 00:54:18   | NaN             | 4.52          | NaN        | 114          |
| 43      | 2        | 2023-01-01 00:44:03  | 2023-01-01 01:13:49   | NaN             | 9.19          | NaN        | 239          |
| 46      | 2        | 2023-01-01 00:50:55  | 2023-01-01 01:19:06   | NaN             | 2.74          | NaN        | 90           |
| ...     | ...      | ...                  | ...                   | ...             | ...           | ...        | ...          |
| 1889059 | 2        | 2023-12-31 23:09:34  | 2023-12-31 23:36:00   | NaN             | 4.10          | NaN        | 140          |
| 1889063 | 2        | 2023-12-31 23:05:26  | 2023-12-31 23:18:01   | NaN             | 3.24          | NaN        | 229          |
| 1889074 | 2        | 2023-12-31 23:38:22  | 2024-01-01 00:02:48   | NaN             | 8.38          | NaN        | 244          |
| 1889076 | 2        | 2023-12-31 23:35:59  | 2023-12-31 23:39:21   | NaN             | 0.76          | NaN        | 141          |
| 1889082 | 1        | 2023-12-31 23:12:30  | 2023-12-31 23:21:12   | NaN             | 0.00          | NaN        | 166          |

64564 rows × 8 columns

There are 64564 rows with Passenger count as NaN. Now, imputing those with the **'mode'** value.

```
df['passenger_count'].fillna(df['passenger_count'].mode()[0], inplace=True)
df['passenger_count'].isnull().sum()

/tmp/ipykernel_4887/736688748.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, in

df['passenger_count'].fillna(df['passenger_count'].mode()[0], inplace=True)
np.int64(0)
```

We have passenger\_count as 0 as well. But logically it is not possible. So I am replacing it with Mode value which is 1.



```
df['passenger_count'].value_counts()
```

```
***
      count
passenger_count
1.0      1436486
2.0      276251
3.0       68768
4.0       38396
0.0       29580
5.0       23775
6.0       15807
8.0         11
9.0          5
7.0          5
```

dtype: int64

```
df.loc[df['passenger_count'] == 0, 'passenger_count'] = df['passenger_count'].mode()[0]
df['passenger_count'].value_counts()
```

```
***
      count
passenger_count
1.0      1466066
```

### 2.2.3. Handle missing values in RatecodeID

#### 2.2.3 [2 marks]

Handle missing values in RatecodeID

```
# Fix missing values in 'RatecodeID'
df['RatecodeID'].isnull().sum()
```

```
np.int64(64564)
```

```
df['RatecodeID'].fillna(df['RatecodeID'].median(), inplace=True)
```

/tmp/ipykernel\_4887/898901588.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through loc or the inplace attribute. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[

```
df['RatecodeID'].fillna(df['RatecodeID'].median(), inplace=True)
```

We have 64564 records with NaN values. I am Imputing those with median value.

## 2.2.4. Impute NaN in congestion\_surcharge

### 2.2.4 [3 marks]

Impute NaN in `congestion_surcharge`

```
52] # handle null values in congestion_surcharge
✓ Os df['congestion_surcharge'].isnull().sum()

np.int64(64564)

53] df['congestion_surcharge'].median()
✓ Os 2.5

54] df['congestion_surcharge'].fillna(df['congestion_surcharge'].median(), inplace=True)
✓ Os ... /tmp/ipykernel_3438/3502353311.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we ar
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col
df['congestion_surcharge'].fillna(df['congestion_surcharge'].median(), inplace=True)
```

There are 64564 rows with NaN RateCodeID. Imputed with median value

---

 # Handle any remaining missing values

```
df.isnull().sum()
```

---

```
***
      0
VendorID      0
tpep_pickup_datetime  0
tpep_dropoff_datetime  0
passenger_count      0
trip_distance      0
RatecodeID      0
PULocationID      0
DOLocationID      0
payment_type      0
fare_amount      0
extra            0
mta_tax         0
tip_amount      0
tolls_amount     0
improvement_surcharge  0
total_amount    10
congestion_surcharge  0
airport_fee      0
```

```
dtype: int64
```

---

 `df.drop((df['total_amount'].isnull()).index, inplace=True)`

---

There are only 10 rows with total\_amount as null which is very small portion of dataset. So I am dropping those rows.

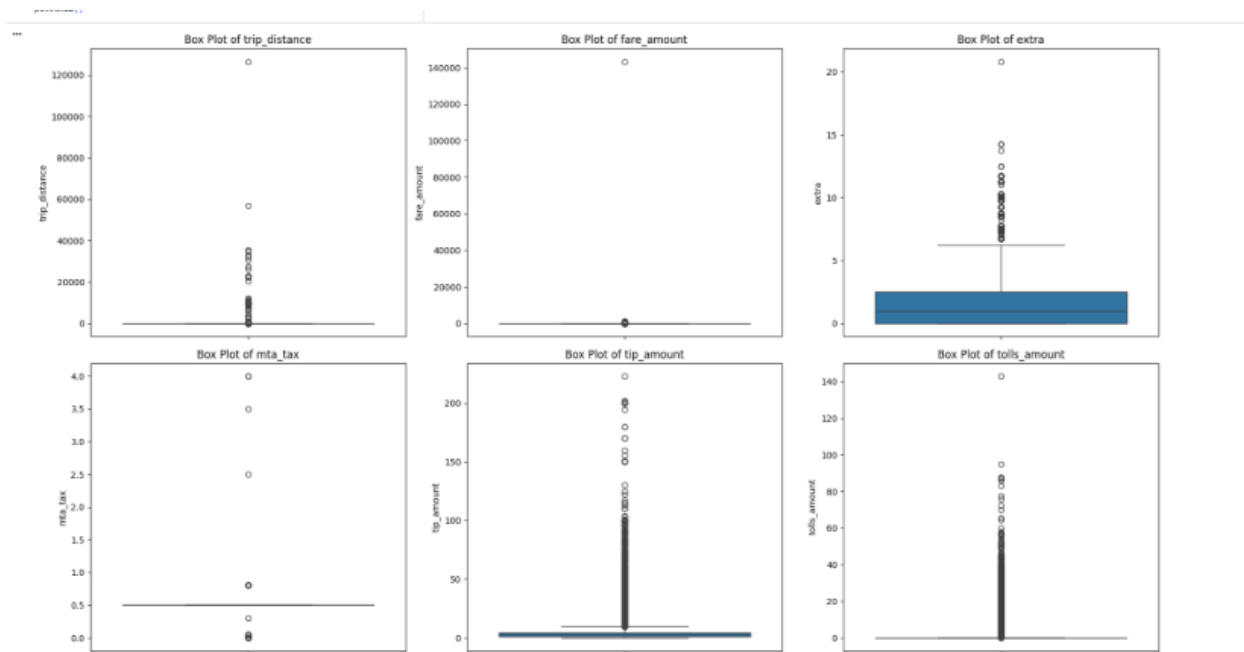
## 2.3. Handling Outliers and Standardising Values

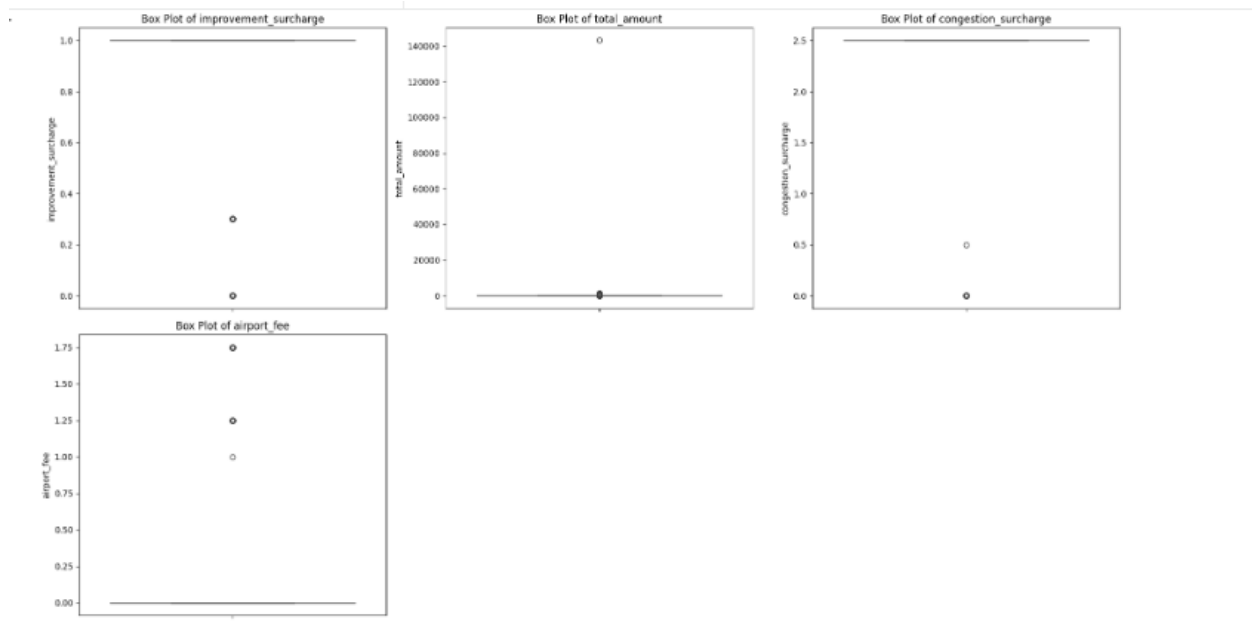
### 2.3.1. Check outliers in payment type, trip distance and tip amount columns

```
# Check for potential out of place values in various columns
df.describe().T
```



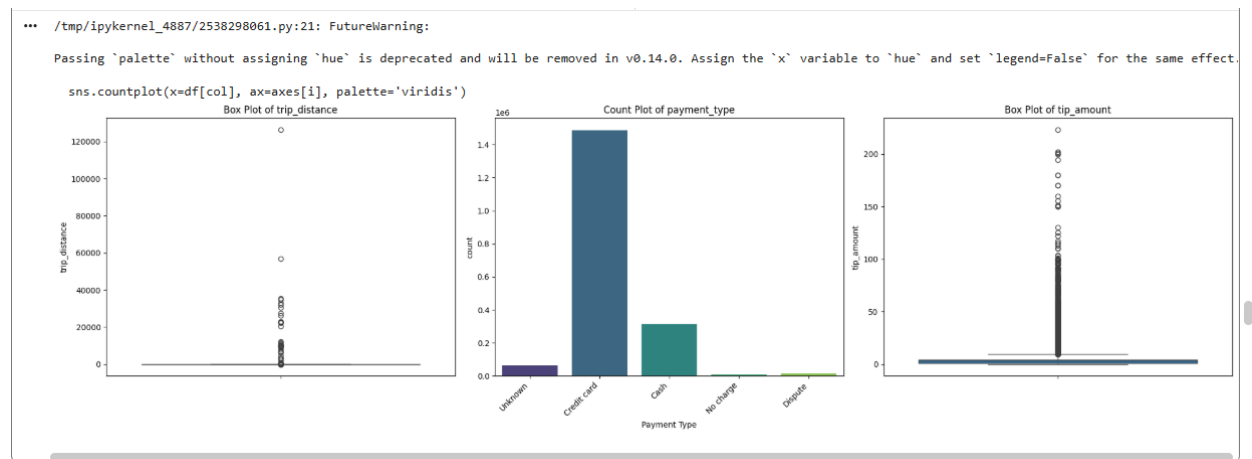
|                       | count     | mean                          | min                    | 25%                           | 50%                           | 75%                           | max                    | std        |
|-----------------------|-----------|-------------------------------|------------------------|-------------------------------|-------------------------------|-------------------------------|------------------------|------------|
| VendorID              | 1889084.0 | 1.733018                      | 1.0                    | 1.0                           | 2.0                           | 2.0                           | 6.0                    | 0.447652   |
| tpep_pickup_datetime  | 1889084   | 2023-07-02<br>20:29:47.321815 | 2022-12-31<br>23:51:30 | 2023-04-02<br>17:04:27.500000 | 2023-06-27<br>16:38:39        | 2023-10-06<br>19:18:16.750000 | 2023-12-31<br>23:57:51 | NaN        |
| tpep_dropoff_datetime | 1889084   | 2023-07-02<br>20:47:13.496013 | 2022-12-31<br>23:56:06 | 2023-04-02<br>17:23:31.250000 | 2023-06-27<br>16:56:18.500000 | 2023-10-06<br>19:35:34.250000 | 2024-01-01<br>20:50:55 | NaN        |
| passenger_count       | 1889084.0 | 1.372274                      | 1.0                    | 1.0                           | 1.0                           | 1.0                           | 9.0                    | 0.864461   |
| trip_distance         | 1889084.0 | 3.860392                      | 0.0                    | 1.05                          | 1.8                           | 3.4                           | 126360.46              | 129.658603 |
| RatecodeID            | 1889084.0 | 1.071882                      | 1.0                    | 1.0                           | 1.0                           | 1.0                           | 6.0                    | 0.387437   |
| PULocationID          | 1889084.0 | 165.280803                    | 1.0                    | 132.0                         | 162.0                         | 234.0                         | 265.0                  | 64.000933  |
| DOLocationID          | 1889084.0 | 164.048275                    | 1.0                    | 114.0                         | 162.0                         | 234.0                         | 265.0                  | 69.802997  |
| payment_type          | 1889084.0 | 1.163817                      | 0.0                    | 1.0                           | 1.0                           | 1.0                           | 4.0                    | 0.508057   |
| fare_amount           | 1889084.0 | 19.922422                     | 0.0                    | 9.3                           | 13.5                          | 21.9                          | 143163.45              | 105.735992 |
| extra                 | 1889084.0 | 1.588041                      | 0.0                    | 0.0                           | 1.0                           | 2.5                           | 20.8                   | 1.829059   |
| mta_tax               | 1889084.0 | 0.495314                      | 0.0                    | 0.5                           | 0.5                           | 0.5                           | 4.0                    | 0.04848    |
| tip_amount            | 1889084.0 | 3.547699                      | 0.0                    | 1.0                           | 2.85                          | 4.42                          | 223.08                 | 4.05571    |
| tolls_amount          | 1889084.0 | 0.596846                      | 0.0                    | 0.0                           | 0.0                           | 0.0                           | 143.0                  | 2.188431   |
| improvement_surcharge | 1889084.0 | 0.999052                      | 0.0                    | 1.0                           | 1.0                           | 1.0                           | 1.0                    | 0.028371   |
| total_amount          | 1889084.0 | 28.986916                     | 0.0                    | 15.96                         | 21.0                          | 30.94                         | 143167.45              | 106.613583 |
| congestion_surcharge  | 1889084.0 | 2.314259                      | 0.0                    | 2.5                           | 2.5                           | 2.5                           | 2.5                    | 0.655631   |
| airport_fee           | 1889084.0 | 0.138025                      | 0.0                    | 0.0                           | 0.0                           | 0.0                           | 1.75                   | 0.457577   |





We have outliers in almost all columns.

The following is the plot for payment\_type, trip\_distance and tip\_amount.



```
▶ anomaly_condition_1 = (df['trip_distance'] < 0.1) & (df['fare_amount'] > 300)
anomaly_condition_1.sum()
```

```
... np.int64(35)
```

35 records with lesser trip\_distance and more fare\_amount. So I am dropping those 35 records.

```
df.shape
```

```
(1889084, 18)
```

```
df.drop(df[anomaly_condition_1].index, inplace=True)
```

```
df.shape
```

```
... (1889049, 18)
```

There are 63 records with trip\_distance and fare\_amount as 0 but the pickup and drop location are different. It is clear that these records are anomalies. So I am dropping those rows.

```
anomaly_condition_2 = (df['trip_distance'] == 0) & (df['fare_amount'] == 0) & (df['PULocationID'] != df['DOLocationID'])  
anomaly_condition_2.sum()
```

```
np.int64(63)
```

```
print(f'Before deletion: {df.shape}')  
df.drop(df[anomaly_condition_2].index, inplace=True)  
print(f'After deletion: {df.shape}')
```

```
... Before deletion: (1889049, 18)  
After deletion: (1888986, 18)
```

Dataset has 46 records where trip distance is more than 250 miles. I am dropping as these are outliers in trip\_distance.

```
anomaly_condition_3 = (df['trip_distance'] > 250)  
anomaly_condition_3.sum()
```

```
np.int64(46)
```

```
print(f'Before deletion: {df.shape}')  
df.drop(df[anomaly_condition_3].index, inplace=True)  
print(f'After deletion: {df.shape}')
```

```
... Before deletion: (1888940, 18)  
/tmp/ipykernel_4887/936822604.py:2: UserWarning: Boolean Series key will be reindexed to match DataFrame index.  
df.drop(df[anomaly_condition_3].index, inplace=True)  
After deletion: (1888940, 18)
```

Since the dataset has payment\_type as 0 for more than 3% records, I am imputing those with mode value.

```
anomaly_condition_4 = (df['payment_type'] == 0)  
anomaly_condition_4.mean()
```

```
np.float64(0.03416413438224613)
```

```
df.loc[df['payment_type'] == 0, 'payment_type'] = df['payment_type'].mode()[0]
```

Removing rows where passenger\_count > 6 since it is not possible in taxi.

```
5 df.shape
(1888940, 18)

5 # remove passenger_count > 6
df.drop(df[df['passenger_count'] > 6].index, inplace=True)

5 df.shape
(1888919, 18)

5 df['passenger_count'].value_counts()
***
count
passenger_count
1.0    1465943
2.0    276237
3.0    68767
4.0    38391
5.0    23775
6.0    15806

dtype: int64
```

### Handling outliers in various columns:

1. Trip\_distance column

```

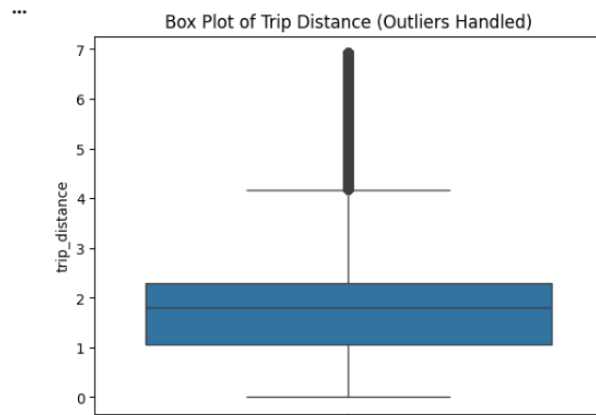
median_trip_distance = df['trip_distance'].median()
Q1_trip_distance = df['trip_distance'].quantile(0.25)
Q3_trip_distance = df['trip_distance'].quantile(0.75)
IQR_trip_distance = Q3_trip_distance - Q1_trip_distance

lower_bound_trip_distance = Q1_trip_distance - 1.5 * IQR_trip_distance
upper_bound_trip_distance = Q3_trip_distance + 1.5 * IQR_trip_distance

df.loc[(df['trip_distance'] < lower_bound_trip_distance) | (df['trip_distance'] > upper_bound_trip_distance), 'trip_distance'] = median_trip_distance

sns.boxplot(y=df['trip_distance'])
plt.title('Box Plot of Trip Distance (Outliers Handled)')
plt.show()

```



## 2. Tip\_amount column

```

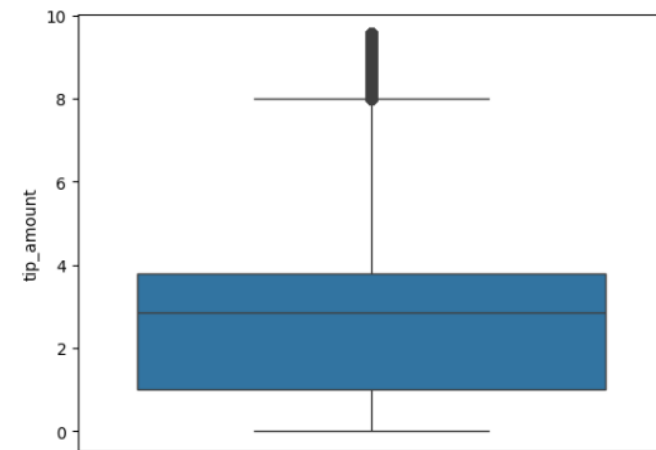
# Calculate the median for 'tip_amount'
median_tip_amount = df['tip_amount'].median()
# Calculate Q1 (25th percentile) and Q3 (75th percentile) for 'tip_amount'
Q1 = df['tip_amount'].quantile(0.25)
Q3 = df['tip_amount'].quantile(0.75)

# Calculate the Interquartile Range (IQR)
IQR = Q3 - Q1
# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Replace outliers (values outside the bounds) with the median
df.loc[(df['tip_amount'] < lower_bound) | (df['tip_amount'] > upper_bound), 'tip_amount'] = median_tip_amount

sns.boxplot(y=df['tip_amount'])
plt.show()

```





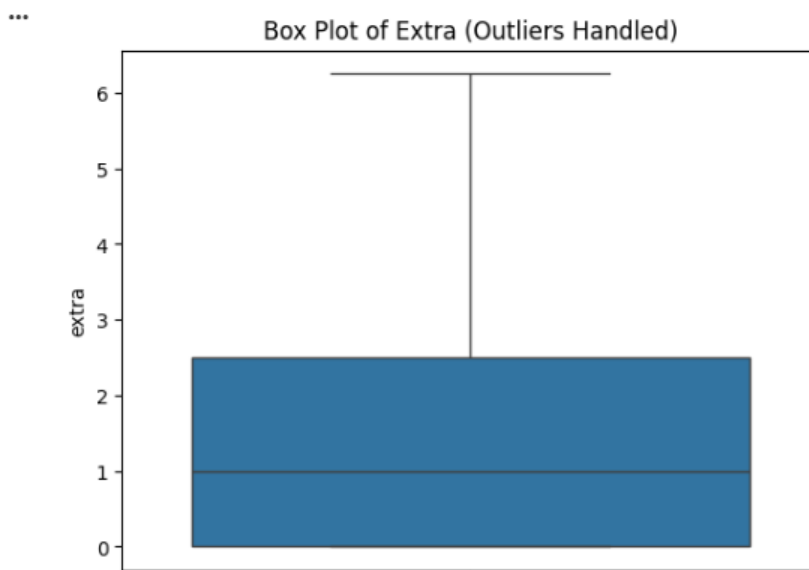
### 3. Extra column

```
median_extra = df['extra'].median()
Q1_extra = df['extra'].quantile(0.25)
Q3_extra = df['extra'].quantile(0.75)
IQR_extra = Q3_extra - Q1_extra

lower_bound_extra = Q1_extra - 1.5 * IQR_extra
upper_bound_extra = Q3_extra + 1.5 * IQR_extra

df.loc[(df['extra'] < lower_bound_extra) | (df['extra'] > upper_bound_extra), 'extra'] = median_extra
```

```
sns.boxplot(y=df['extra'])
plt.title('Box Plot of Extra (Outliers Handled)')
plt.show()
```



### 4. Mta\_tax column

```

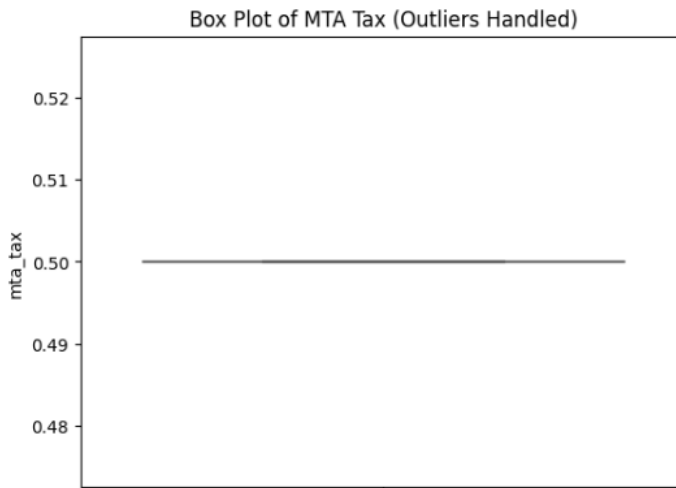
median_mta_tax = df['mta_tax'].median()
Q1_mta_tax = df['mta_tax'].quantile(0.25)
Q3_mta_tax = df['mta_tax'].quantile(0.75)
IQR_mta_tax = Q3_mta_tax - Q1_mta_tax

lower_bound_mta_tax = Q1_mta_tax - 1.5 * IQR_mta_tax
upper_bound_mta_tax = Q3_mta_tax + 1.5 * IQR_mta_tax

df.loc[(df['mta_tax'] < lower_bound_mta_tax) | (df['mta_tax'] > upper_bound_mta_tax), 'mta_tax'] = median_mta_tax

sns.boxplot(y=df['mta_tax'])
plt.title('Box Plot of MTA Tax (Outliers Handled)')
plt.show()

```



## 5. Toll\_amount column

```

median_tolls_amount = df['tolls_amount'].median()
Q1_tolls_amount = df['tolls_amount'].quantile(0.25)
Q3_tolls_amount = df['tolls_amount'].quantile(0.75)
IQR_tolls_amount = Q3_tolls_amount - Q1_tolls_amount

lower_bound_tolls_amount = Q1_tolls_amount - 1.5 * IQR_tolls_amount
upper_bound_tolls_amount = Q3_tolls_amount + 1.5 * IQR_tolls_amount

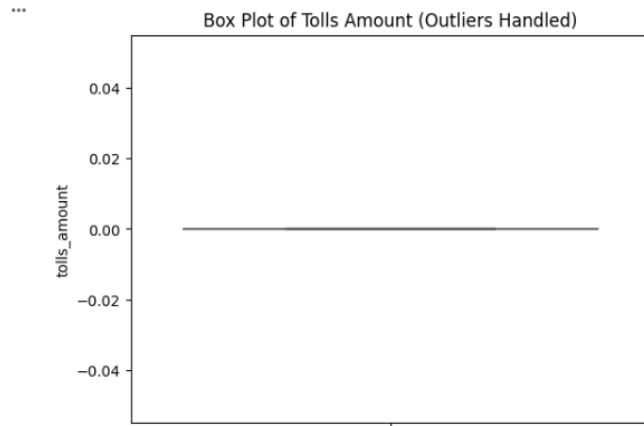
df.loc[(df['tolls_amount'] < lower_bound_tolls_amount) | (df['tolls_amount'] > upper_bound_tolls_amount), 'tolls_amount'] = median_tolls_amount

```

```

sns.boxplot(y=df['tolls_amount'])
plt.title('Box Plot of Tolls Amount (Outliers Handled)')
plt.show()

```



### 3. Exploratory Data Analysis

#### 3.1. General EDA: Finding Patterns and Trends

##### 3.1.1. Classify variables into categorical and numerical

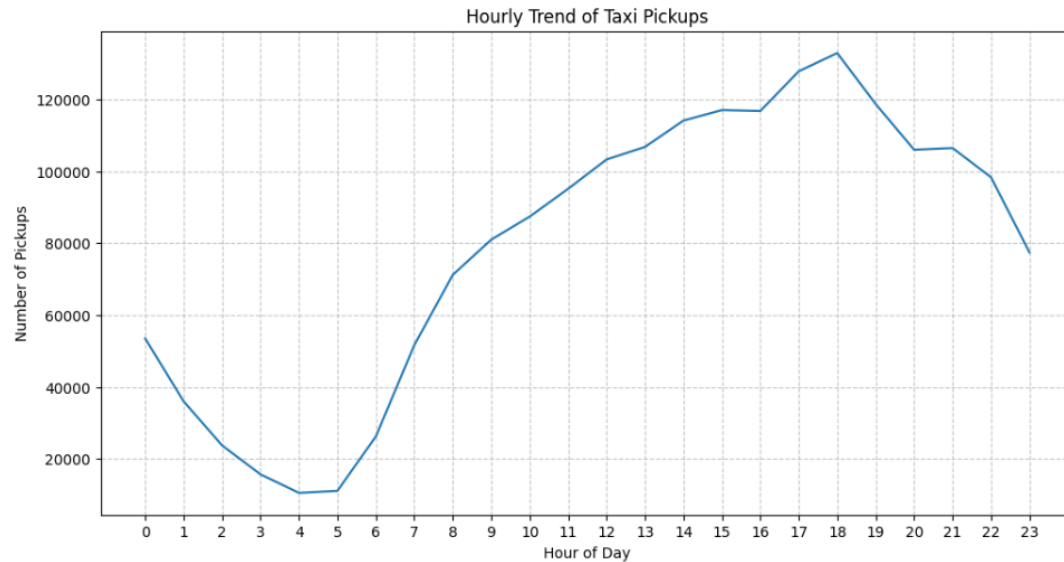
**Numerical columns:** VendorID, trip\_distance, PULocationID, DOLocationID, trip\_duration, pickup\_hour

**Categorical columns:** passenger\_count, RatecodeID, payment\_type

**Datetime columns:** tpep\_pickup\_datetime, tpep\_dropoff\_datetime

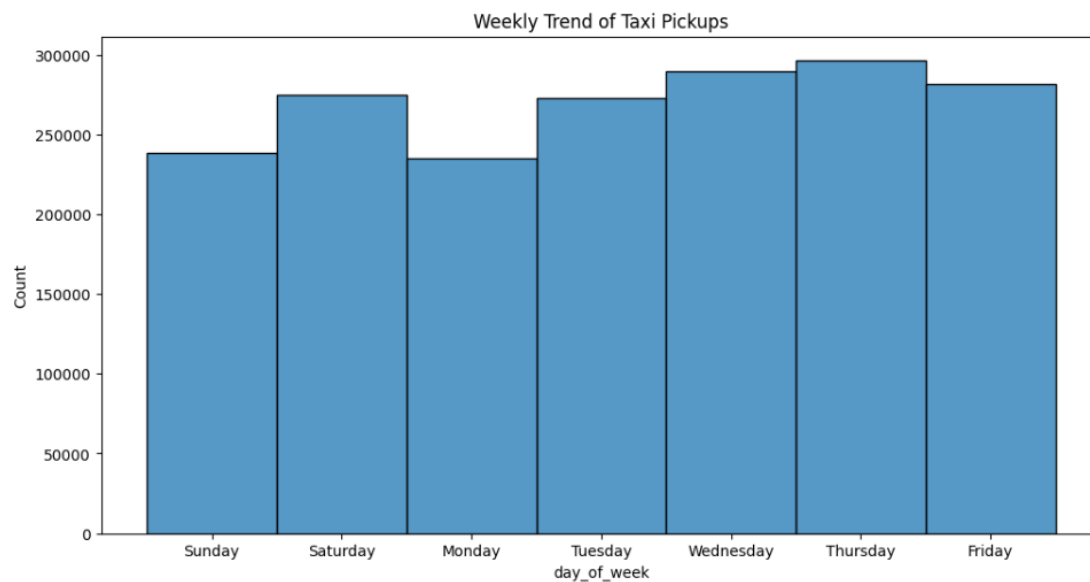
##### 3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

```
plt.figure(figsize=(12, 6))
sns.lineplot(x=trend_data.index, y=trend_data.values)
plt.title('Hourly Trend of Taxi Pickups')
plt.xlabel('Hour of Day')
plt.ylabel('Number of Pickups')
plt.xticks(range(0, 24)) # Ensure all hours are displayed on x-axis
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



we have more pickups at 6pm and less pickup at 4am. From 4am to 6pm, the number of pickups is gradually increasing. From 6pm to 4am, the number of pickups is gradually decreasing.

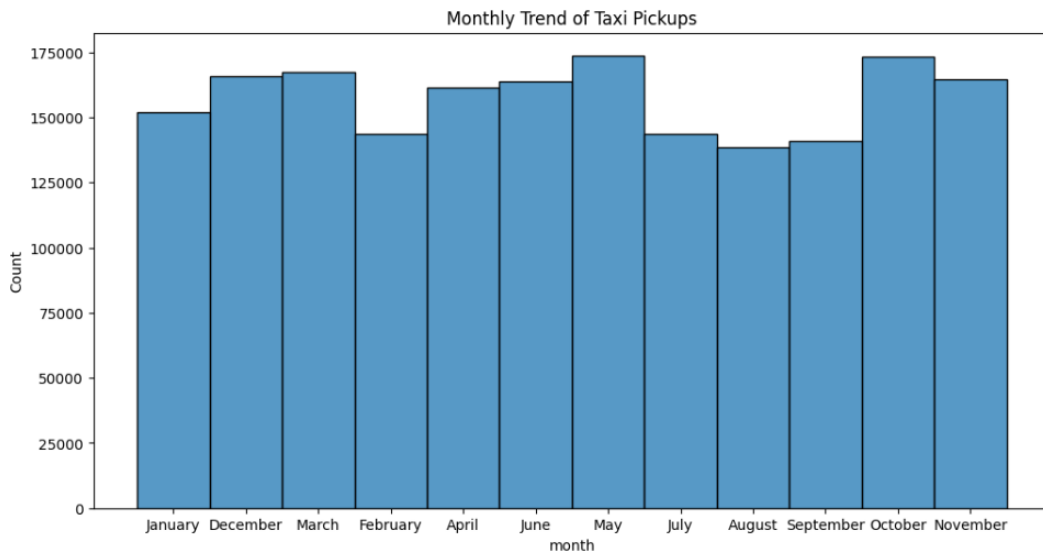
```
plt.figure(figsize=(12, 6))
plt.title('Weekly Trend of Taxi Pickups')
sns.histplot(data=df, x="day_of_week")
plt.show()
```



We have less pickups on Sunday and Monday. The highest number of pickups is on Thursday

```
# Show the monthly trends in pickups
```

```
plt.figure(figsize=(12, 6))  
plt.title('Monthly Trend of Taxi Pickups')  
sns.histplot(data=df, x="month")  
plt.show()
```



Quarter 3 has less number of pickups. May month has the highest number of pickups.

### 3.1.3. Filter out the zero/negative values in fares, distance and tips

```
# Create a df with non zero entries for the selected parameters.  
df_non_zero = df[(df['fare_amount'] != 0) & (df['tip_amount'] != 0) & (df['total_amount'] != 0) & (df['trip_distance'] != 0)]  
print(df_non_zero.shape)
```

(1445731, 21)

Filtered out around 4,43,188 records since it has zero values for some columns.

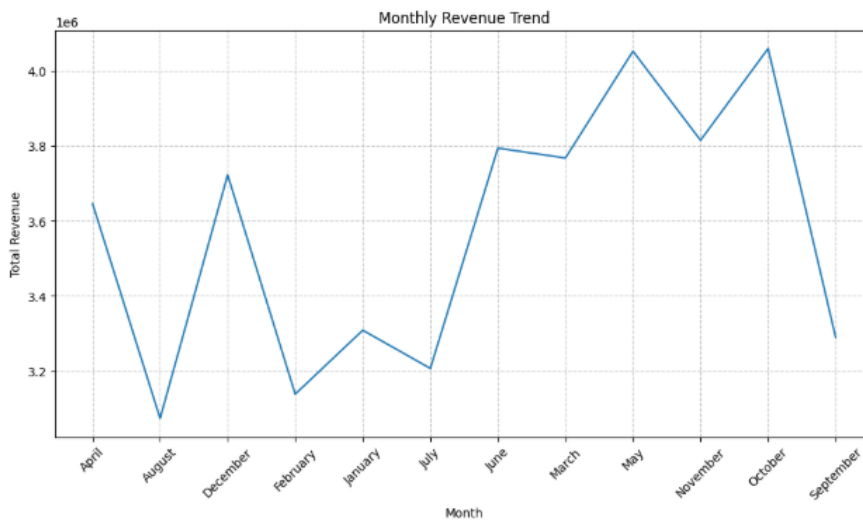
### 3.1.4. Analyse the monthly revenue trends

```
# Group data by month and analyse monthly revenue
```

```
monthly_revenue = df_non_zero.groupby('month')['total_amount'].sum()  
print(monthly_revenue)
```

```
month  
April      3646175.77  
August     3873852.35  
December   3722941.51  
February   3137773.89  
January     3388386.14  
July       3286444.76  
June       3794268.24  
March      3767843.58  
May        4852873.83  
November   3815114.23  
October    4859933.59  
September  3289125.02  
Name: total_amount, dtype: float64
```

```
plt.figure(figsize=(12, 6))  
sns.lineplot(x=monthly_revenue.index, y=monthly_revenue.values)  
plt.title('Monthly Revenue Trend')  
plt.xlabel('Month')  
plt.ylabel('Total Revenue')  
plt.xticks(rotation=45) # Rotate x-axis labels for better readability  
plt.grid(True, linestyle='--', alpha=0.7)  
plt.show()
```



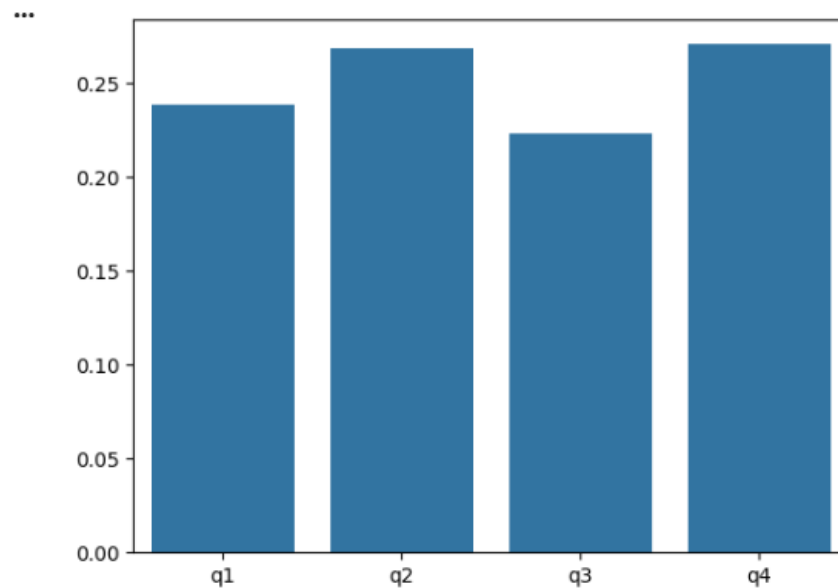
We earned more revenue on October and May and less on August.

### 3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

```
print(f"Proportion of Q1: {q1}")  
print(f"Proportion of Q2: {q2}")  
print(f"Proportion of Q3: {q3}")  
print(f"Proportion of Q4: {q4}")
```

```
Proportion of Q1: 0.23823204173642204  
Proportion of Q2: 0.26807275774138184  
Proportion of Q3: 0.22318087286284405  
Proportion of Q4: 0.27051432765935257
```

```
sns.barplot(x=['q1', 'q2', 'q3', 'q4'], y=[q1, q2, q3, q4])  
plt.show()
```



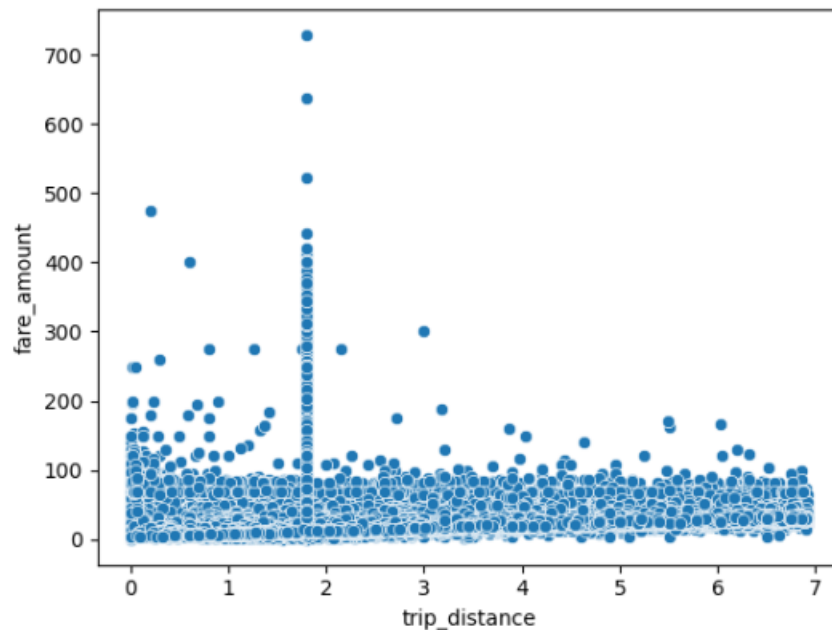
We have more revenue on q4 and less on q3. Almost q2 and q4 are same.

### 3.1.6. Analyse and visualise the relationship between distance and fare amount

▶ # Show how trip fare is affected by distance

```
sns.scatterplot(data=df_non_zero, x='trip_distance', y='fare_amount')  
plt.show()
```

...



▶ `df_non_zero['trip_distance'].corr(df_non_zero['fare_amount'])`

... `np.float64(0.28224678961871774)`

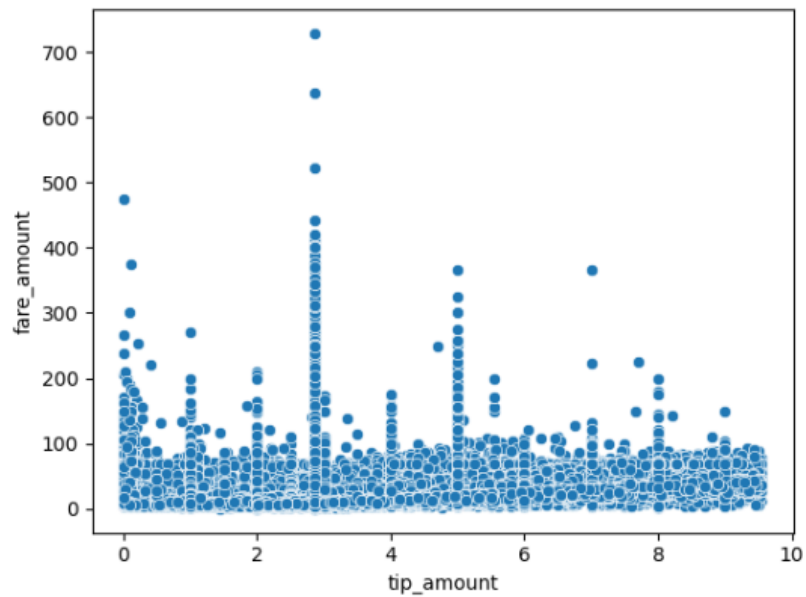
There is some positive correlation of 0.28.

### 3.1.7. Analyse the relationship between fare/tips and trips/passengers

Fare amount and tip amount correlation



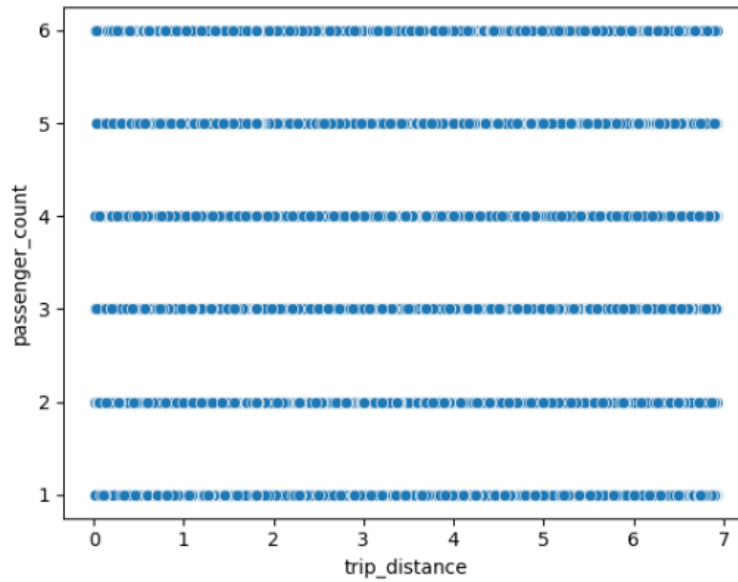
```
sns.scatterplot(data=df_non_zero, x='tip_amount', y='fare_amount')  
plt.show()
```



```
df_non_zero['tip_amount'].corr(df_non_zero['fare_amount'])  
... np.float64(0.23829087361157675)
```

Trip\_distance and Passenger\_count correlation – there is no correlation

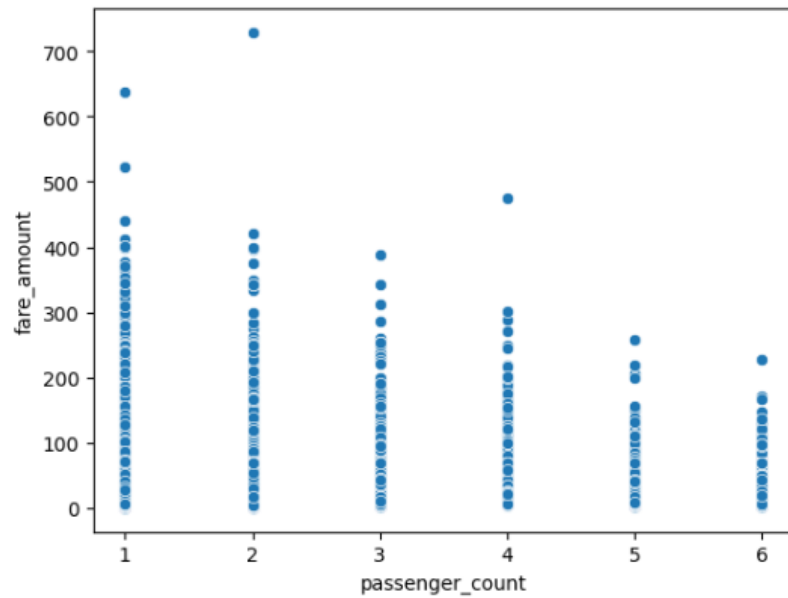
```
sns.scatterplot(data=df_non_zero, x='trip_distance', y='passenger_count')  
plt.show()
```



```
df_non_zero['trip_distance'].corr(df_non_zero['passenger_count'])  
... np.float64(0.008028362552324378)
```

There is no significant correlation between fare amount and passenger count.

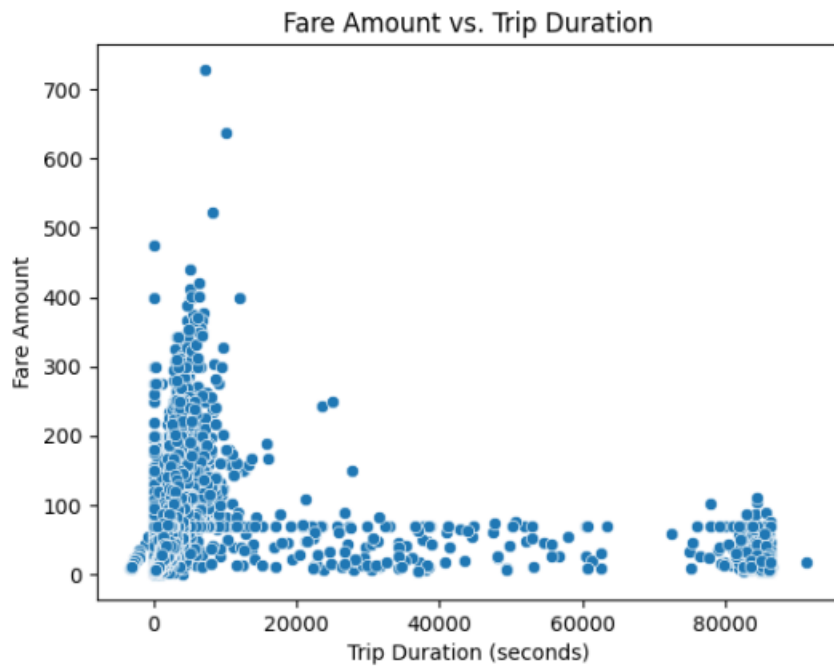
```
sns.scatterplot(data=df_non_zero, x='passenger_count', y='fare_amount')  
plt.show()
```



```
df_non_zero['passenger_count'].corr(df_non_zero['fare_amount'])  
np.float64(0.035866325198731716)
```

There is a positive correlation of 0.33 between fare\_amount and trip duration.

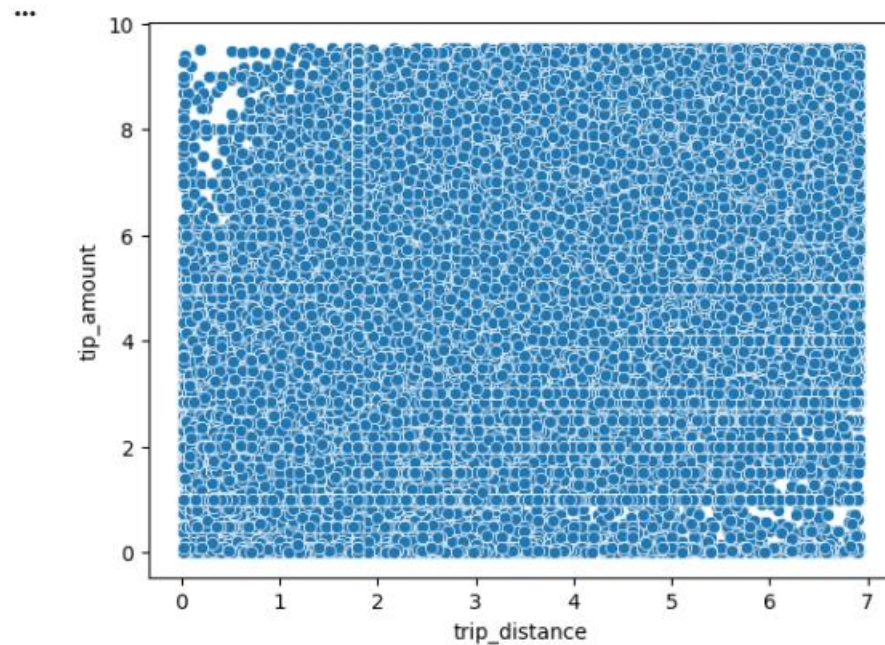
```
df_non_zero['trip_duration_seconds'] = df_non_zero['trip_duration'].dt.total_seconds()
```



Correlation between fare\_amount and trip\_duration: 0.3323093188319888

There is a significant positive correlation of 0.5 between trip distance and tip amount.

```
# Show relationship between tip and trip distance
sns.scatterplot(data=df_non_zero, x='trip_distance', y='tip_amount')
plt.show()
correlation = df_non_zero['trip_distance'].corr(df_non_zero['tip_amount'])
print(f"Correlation between trip_distance and tip_amount: {correlation}")
```

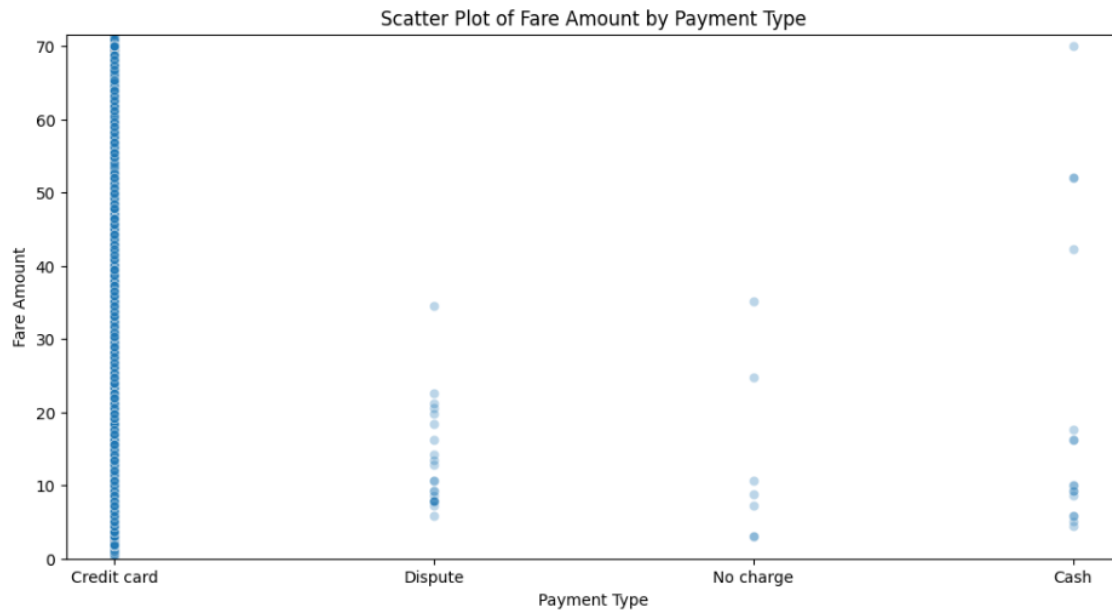


Correlation between trip\_distance and tip\_amount: 0.5332116824417185

### 3.1.8. Analyse the distribution of different payment types

```
# Analyse the distribution of different payment types (payment_type).

plt.figure(figsize=(12, 6))
sns.scatterplot(data=df_non_zero, x='payment_type', y='fare_amount', alpha=0.3)
plt.title('Scatter Plot of Fare Amount by Payment Type')
plt.xlabel('Payment Type')
plt.ylabel('Fare Amount')
# Limiting y-axis to better visualize the main distribution, as there are high outliers
plt.ylim(0, df_non_zero['fare_amount'].quantile(0.99))
plt.xticks(ticks=df_non_zero['payment_type'].unique(), labels={1: 'Credit card', 2: 'Cash', 4: 'Dispute', 3: 'No charge'}.values())
plt.show()
```



```
df_non_zero['payment_type'].value_counts()
```

```
...
      count
payment_type
1      1445685
2         23
4         16
3          7

dtype: int64
```

We have more credit card payments only. Other payments are too small.

### 3.1.9. Load the taxi zones shapefile and display it

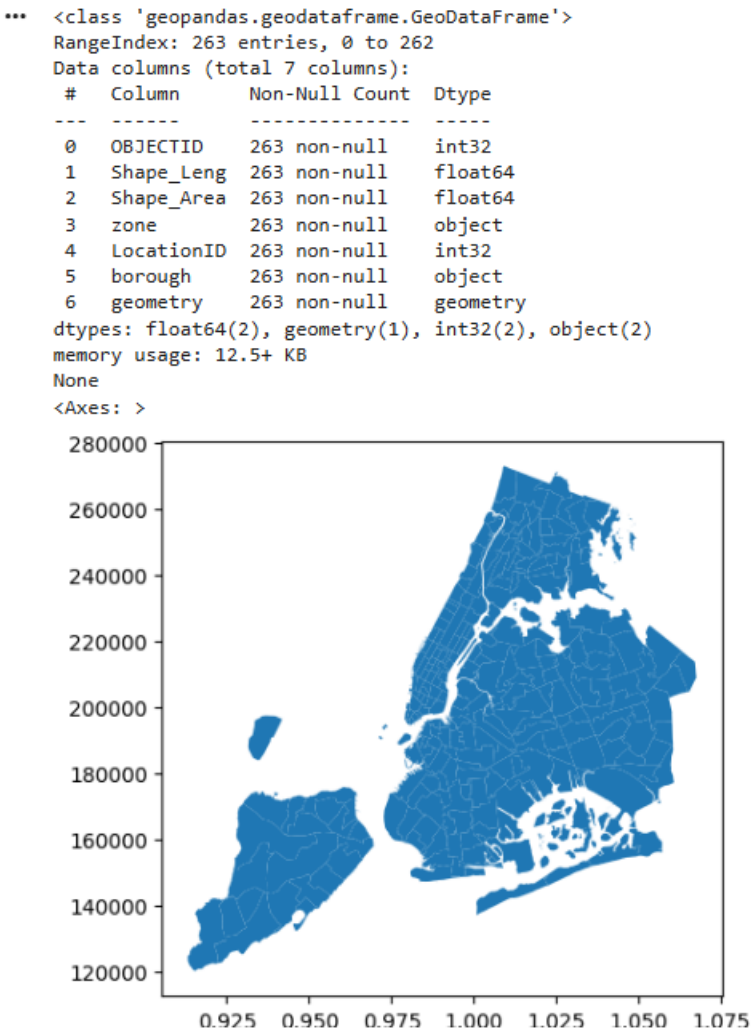
```
import geopandas as gpd

# Read the shapefile using geopandas
zones = gpd.read_file('taxi_zones.shp') # read the .shp file using gpd
zones.head()
```

|   | OBJECTID | Shape_Leng | Shape_Area | zone                    | LocationID | borough       | geometry                                          |
|---|----------|------------|------------|-------------------------|------------|---------------|---------------------------------------------------|
| 0 | 1        | 0.116357   | 0.000782   | Newark Airport          | 1          | EWR           | POLYGON ((933100.918 192536.086, 933091.011 19... |
| 1 | 2        | 0.433470   | 0.004866   | Jamaica Bay             | 2          | Queens        | MULTIPOLYGON (((1033269.244 172126.008, 103343... |
| 2 | 3        | 0.084341   | 0.000314   | Allerton/Pelham Gardens | 3          | Bronx         | POLYGON ((1026308.77 256767.698, 1026495.593 2... |
| 3 | 4        | 0.043567   | 0.000112   | Alphabet City           | 4          | Manhattan     | POLYGON ((992073.467 203714.076, 992068.667 20... |
| 4 | 5        | 0.092146   | 0.000498   | Arden Heights           | 5          | Staten Island | POLYGON ((935843.31 144283.336, 936046.565 144... |

Next steps: [Generate code with zones](#) [New interactive sheet](#)

```
print(zones.info())
zones.plot()
```



### 3.1.10. Merge the zone data with trips data

12]

2s

# Merge zones and trip records using locationID and PULocationID

merged = pd.merge(df\_non\_zero, zones, left\_on='PULocationID', right\_on='LocationID')

merged.head()

...

|   | VendorID | tpep_pickup_datetime | tpep_dropoff_datetime | passenger_count | trip_distance | RatecodeID | PULocationID | DOLocationID | payment_type | fare_ |
|---|----------|----------------------|-----------------------|-----------------|---------------|------------|--------------|--------------|--------------|-------|
| 0 | 2        | 2023-01-01 00:16:41  | 2023-01-01 00:21:46   | 2.0             | 1.24          | 1.0        | 161          | 237          | 1            |       |
| 1 | 2        | 2023-01-01 00:43:00  | 2023-01-01 01:01:00   | 1.0             | 1.80          | 1.0        | 66           | 107          | 1            |       |
| 2 | 1        | 2023-01-01 00:42:56  | 2023-01-01 01:16:33   | 2.0             | 1.80          | 1.0        | 246          | 37           | 1            |       |
| 3 | 2        | 2023-01-01 00:58:00  | 2023-01-01 01:08:31   | 2.0             | 1.59          | 1.0        | 79           | 164          | 1            |       |
| 4 | 2        | 2023-01-01 00:16:06  | 2023-01-01 00:31:59   | 1.0             | 3.16          | 1.0        | 79           | 256          | 1            |       |

5 rows × 30 columns

### 3.1.11. Find the number of trips for each zone/location ID



```
# Group data by location and calculate the number of trips

groupby_location = merged.groupby('LocationID').size().reset_index(name='total_trips')
```

groupby\_location

| ... | LocationID | total_trips |  |
|-----|------------|-------------|--|
| 0   | 1          | 37          |  |
| 1   | 2          | 2           |  |
| 2   | 4          | 1690        |  |
| 3   | 6          | 1           |  |
| 4   | 7          | 356         |  |
| ... | ...        | ...         |  |
| 227 | 259        | 1           |  |
| 228 | 260        | 111         |  |
| 229 | 261        | 7077        |  |
| 230 | 262        | 20448       |  |
| 231 | 263        | 28716       |  |

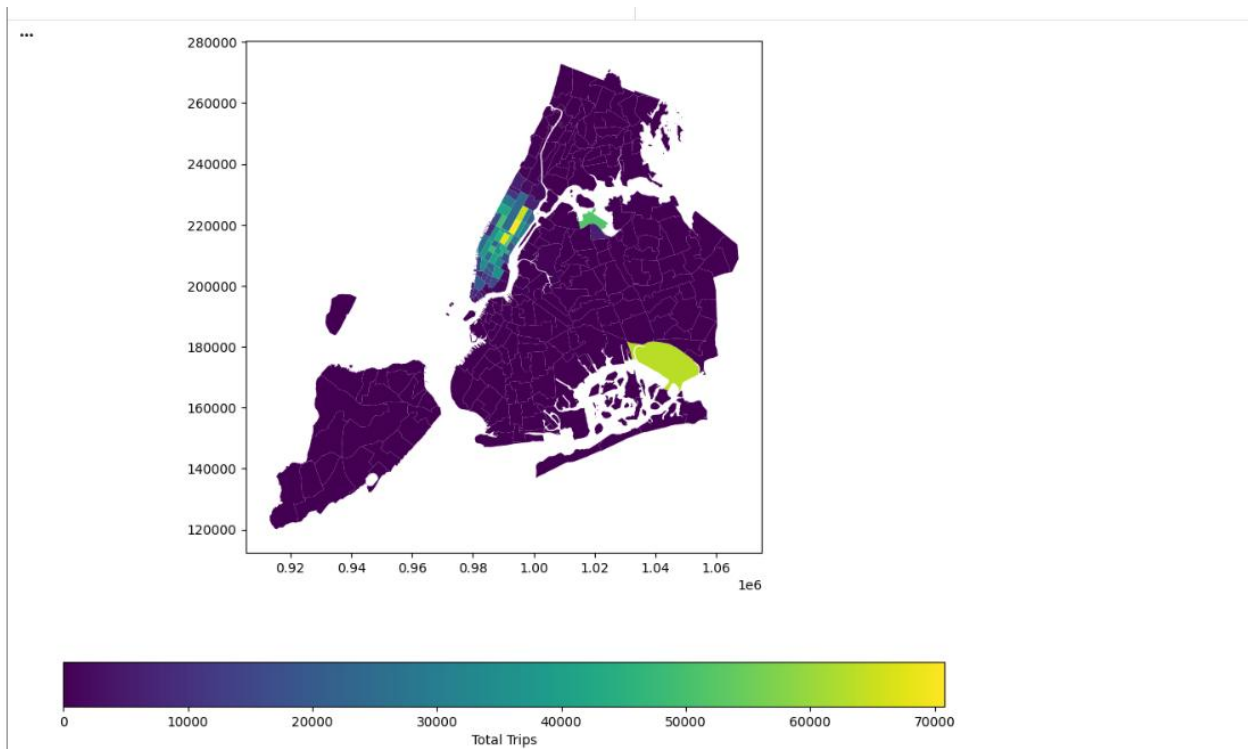
232 rows × 2 columns

### 3.1.12. Add the number of trips for each zone to the zones dataframe

```
# Merge trip counts back to the zones GeoDataFrame
zones_with_trips = zones.merge(groupby_location, on='LocationID', how='left')
zones_with_trips['total_trips'] = zones_with_trips['total_trips'].fillna(0).astype(int)
display(zones_with_trips.head())
```

| ... | OBJECTID | Shape_Leng | Shape_Area | zone                    | LocationID | borough       | geometry                                          | total_trips |  |
|-----|----------|------------|------------|-------------------------|------------|---------------|---------------------------------------------------|-------------|--|
| 0   | 1        | 0.116357   | 0.000782   | Newark Airport          | 1          | EWB           | POLYGON ((933100.918 192536.086, 933091.011 19... | 37          |  |
| 1   | 2        | 0.433470   | 0.004866   | Jamaica Bay             | 2          | Queens        | MULTIPOLYGON (((1033269.244 172126.008, 103343... | 2           |  |
| 2   | 3        | 0.084341   | 0.000314   | Allerton/Pelham Gardens | 3          | Bronx         | POLYGON ((1026308.77 256767.698, 1026495.593 2... | 0           |  |
| 3   | 4        | 0.043567   | 0.000112   | Alphabet City           | 4          | Manhattan     | POLYGON ((992073.467 203714.076, 992068.667 20... | 1690        |  |
| 4   | 5        | 0.092146   | 0.000498   | Arden Heights           | 5          | Staten Island | POLYGON ((935843.31 144283.336, 936046.565 144... | 0           |  |

### 3.1.13. Plot a map of the zones showing number of trips



### 3.1.14. Conclude with results

Upper East Side South zone has highest number of trips and also we have 30 zones with zero trips.

zones\_with\_trips[zones\_with\_trips['total\_trips'] == 0]

| OBJECTID | Shape_Leng | Shape_Area | zone     | LocationID                                    | borough | geometry      | total_trips                                       |   |
|----------|------------|------------|----------|-----------------------------------------------|---------|---------------|---------------------------------------------------|---|
| 2        | 3          | 0.084341   | 0.000314 | Allerton/Pelham Gardens                       | 3       | Bronx         | POLYGON ((1026308.77 256767.698, 1026495.593 2... | 0 |
| 4        | 5          | 0.092146   | 0.000498 | Arden Heights                                 | 5       | Staten Island | POLYGON ((935843.31 144283.336, 936046.565 144... | 0 |
| 26       | 27         | 0.202509   | 0.001341 | Breezy Point/Fort Tilden/Riis Beach           | 27      | Queens        | POLYGON ((1021692.969 147138.664, 1021883.624 ... | 0 |
| 29       | 30         | 0.094510   | 0.000146 | Broad Channel                                 | 30      | Queens        | POLYGON ((1034013.272 163473.152, 1034866.459 ... | 0 |
| 31       | 32         | 0.054267   | 0.000151 | Bronxdale                                     | 32      | Bronx         | POLYGON ((1023302.25 251923.918, 1022481.96 25... | 0 |
| 43       | 44         | 0.235689   | 0.001945 | Charleston/Tottenville                        | 44      | Staten Island | POLYGON ((925284.273 142075.259, 924921.951 14... | 0 |
| 57       | 58         | 0.059855   | 0.000205 | Country Club                                  | 58      | Bronx         | POLYGON ((1035263.069 248349.465, 1035243.096 ... | 0 |
| 83       | 84         | 0.233624   | 0.002074 | Eltingville/Annadale/Prince's Bay             | 84      | Staten Island | POLYGON ((939754.454 131548.91, 939802.804 131... | 0 |
| 85       | 86         | 0.134245   | 0.000623 | Far Rockaway                                  | 86      | Queens        | POLYGON ((1049025.346 163141.969, 1049041.462 ... | 0 |
| 93       | 94         | 0.049833   | 0.000063 | Fordham South                                 | 94      | Bronx         | POLYGON ((1012009.465 253420.02, 1012066.459 2... | 0 |
| 95       | 96         | 0.185180   | 0.000548 | Forest Park/Highland Park                     | 96      | Queens        | POLYGON ((1029152.314 197379.274, 1029238.052 ... | 0 |
| 98       | 99         | 0.183371   | 0.001210 | Freshkills Park                               | 99      | Staten Island | POLYGON ((937468.341 152998.406, 937320.263 15... | 0 |
| 102      | 103        | 0.014306   | 0.000006 | Governor's Island/Ellis Island/Liberty Island | 103     | Manhattan     | POLYGON ((972079.575 190732.429, 972184.766 19... | 0 |
| 103      | 104        | 0.021221   | 0.000012 | Governor's Island/Ellis Island/Liberty Island | 103     | Manhattan     | POLYGON ((973172.666 194632.348, 973310.63 194... | 0 |
| 104      | 105        | 0.077425   | 0.000369 | Governor's Island/Ellis Island/Liberty Island | 103     | Manhattan     | POLYGON ((979605.759 191880.575, 979978.435 19... | 0 |
| 108      | 109        | 0.178268   | 0.001170 | Great Kills                                   | 109     | Staten Island | POLYGON ((943392.585 147162.426, 943412.636 14... | 0 |
| 109      | 110        | 0.103946   | 0.000526 | Great Kills Park                              | 110     | Staten Island | POLYGON ((951420.092 139894.411, 951856.209 13... | 0 |
| 114      | 115        | 0.116169   | 0.000373 | Grymes Hill/Clifton                           | 115     | Staten Island | POLYGON ((961850.466 167915.309, 961831.926 16... | 0 |
| 121      | 122        | 0.066532   | 0.000227 | Hollis                                        | 122     | Queens        | POLYGON ((1051539.66 201007.013, 1051920.726 2... | 0 |
| 146      | 147        | 0.058765   | 0.000106 | Longwood                                      | 147     | Bronx         | POLYGON ((1013266.371 240896.958, 1013354.947 ... | 0 |
| 171      | 172        | 0.118476   | 0.000658 | New Dorp/Midland Beach                        | 172     | Staten Island | POLYGON ((960204.812 146820.751, 960103.437 14... | 0 |
| 175      | 176        | 0.151995   | 0.000658 | Oakwood                                       | 176     | Staten Island | POLYGON ((950393.94 148827.195, 950393.983 148... | 0 |
| 182      | 183        | 0.039826   | 0.000095 | Pelham Bay                                    | 183     | Bronx         | POLYGON ((1029414.164 246587.728, 1029380.19 2... | 0 |
| 186      | 187        | 0.126868   | 0.000421 | Port Richmond                                 | 187     | Staten Island | MULTIPOLYGON (((946964.115 173161.64, 946977.8... | 0 |
| 200      | 201        | 0.130404   | 0.000619 | Rockaway Park                                 | 201     | Queens        | POLYGON ((1034661.735 153320.591, 1034718.155 ... | 0 |
| 203      | 204        | 0.101912   | 0.000641 | Rossville/Woodrow                             | 204     | Staten Island | POLYGON ((927766.539 141772.465, 927864.475 14... | 0 |

Few insights which we got from above analysis are as follows:

- Busiest hours, days and months:  
Hour – 6pm  
Day – Thursday  
Month – October and May
- Trends in revenue collected  
October month earned more revenue
- Trends in quarterly revenue  
More revenue on quarter 4.
- How fare depends on trip distance, trip duration and passenger counts  
There is a positive correlation between fare amount with almost all other columns.
- How tip amount depends on trip distance  
Positive correlation of 0.5

Busiest zones are Upper East Side South, Midtown Center, Upper East Side North, JFK Airport

## 3.2. Detailed EDA: Insights and Strategies

### 3.2.1. Identify slow routes by comparing average speeds on different routes

```
route_speed = merged_df.groupby(['PULocationID', 'DOLocationID', 'hour'])['speed'].mean().reset_index()
slowest_routes = route_speed.sort_values(by='speed').head(10)
display(slowest_routes)
```

|       | PULocationID | DOLocationID | hour | speed     |
|-------|--------------|--------------|------|-----------|
| 33352 | 114          | 120          | 1    | -0.002490 |
| 33252 | 114          | 106          | 1    | -0.001361 |
| 75223 | 198          | 36           | 1    | -0.000539 |
| 96985 | 243          | 264          | 17   | 0.000002  |
| 6210  | 43           | 10           | 10   | 0.000006  |
| 7719  | 45           | 45           | 10   | 0.000017  |
| 63859 | 162          | 162          | 4    | 0.000018  |
| 3044  | 22           | 22           | 15   | 0.000020  |
| 92138 | 237          | 57           | 10   | 0.000021  |
| 95144 | 238          | 257          | 16   | 0.000021  |

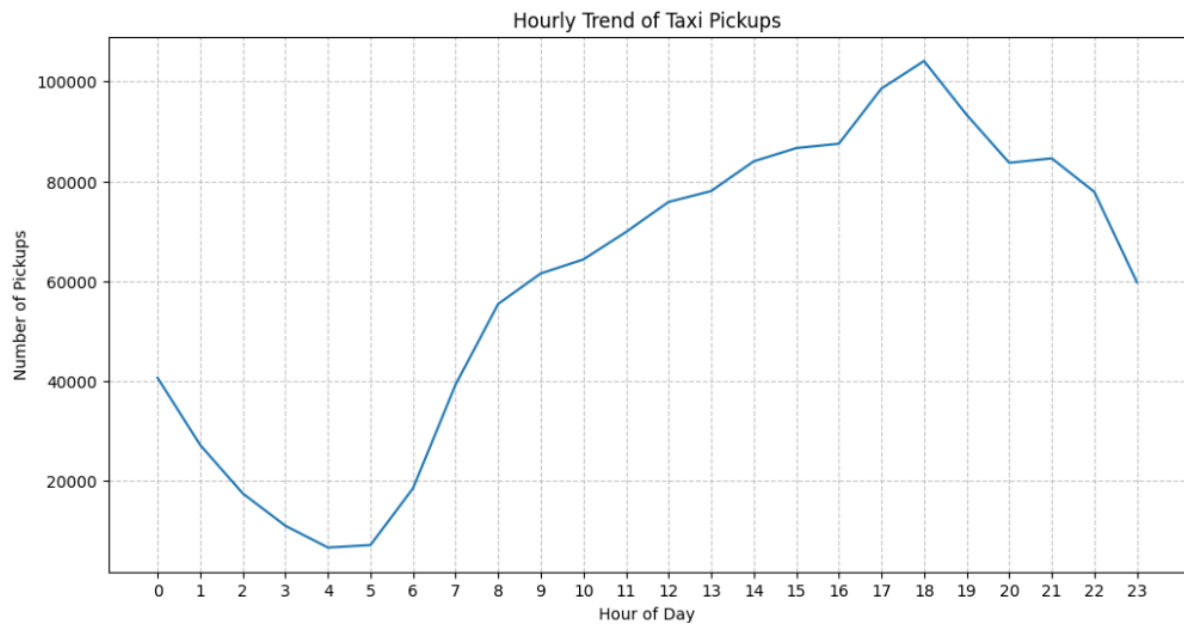
```
merged_df[['PULocationID', 'total_trips']].drop_duplicates().sort_values(by='total_trips', ascending=False).head(10)
```

|     | PULocationID | total_trips |
|-----|--------------|-------------|
| 27  | 237          | 70823       |
| 0   | 161          | 68447       |
| 47  | 236          | 65021       |
| 5   | 132          | 63598       |
| 25  | 162          | 53214       |
| 109 | 138          | 51661       |
| 9   | 142          | 49868       |
| 92  | 186          | 48951       |
| 564 | 230          | 44969       |
| 14  | 170          | 43839       |

High demand zones are 237,161,236 and so on.

### 3.2.2. Calculate the hourly number of trips and identify the busy hours

The busiest hour is 18:00 with 104074 trips.



### 3.2.3. Scale up the number of trips from above to find the actual number of trips

### 3.2.3 [2 mark]

Find the actual number of trips in the five busiest hours

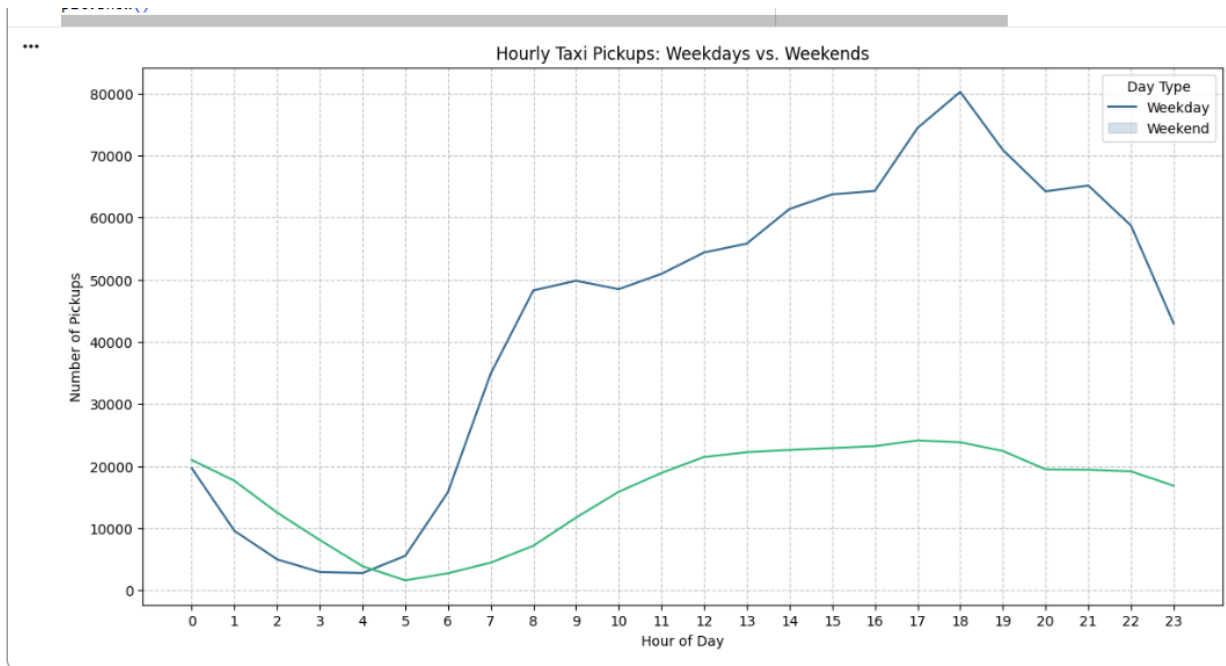
```
# Scale up the number of trips
```

```
# Fill in the value of your sampling fraction and use that to scale up the numbers  
sample_fraction = 0.05
```

```
# Find the five busiest hours and their actual number of trips  
busiest_hours_sampled = trend_data.nlargest(5)  
actual_busiest_hours = busiest_hours_sampled / sample_fraction  
  
print("Top 5 busiest hours and their actual number of trips:")  
print(actual_busiest_hours)
```

```
*** Top 5 busiest hours and their actual number of trips:  
hour  
18    2081480.0  
17    1971420.0  
19    1866300.0  
16    1750300.0  
15    1732920.0  
Name: hour, dtype: float64
```

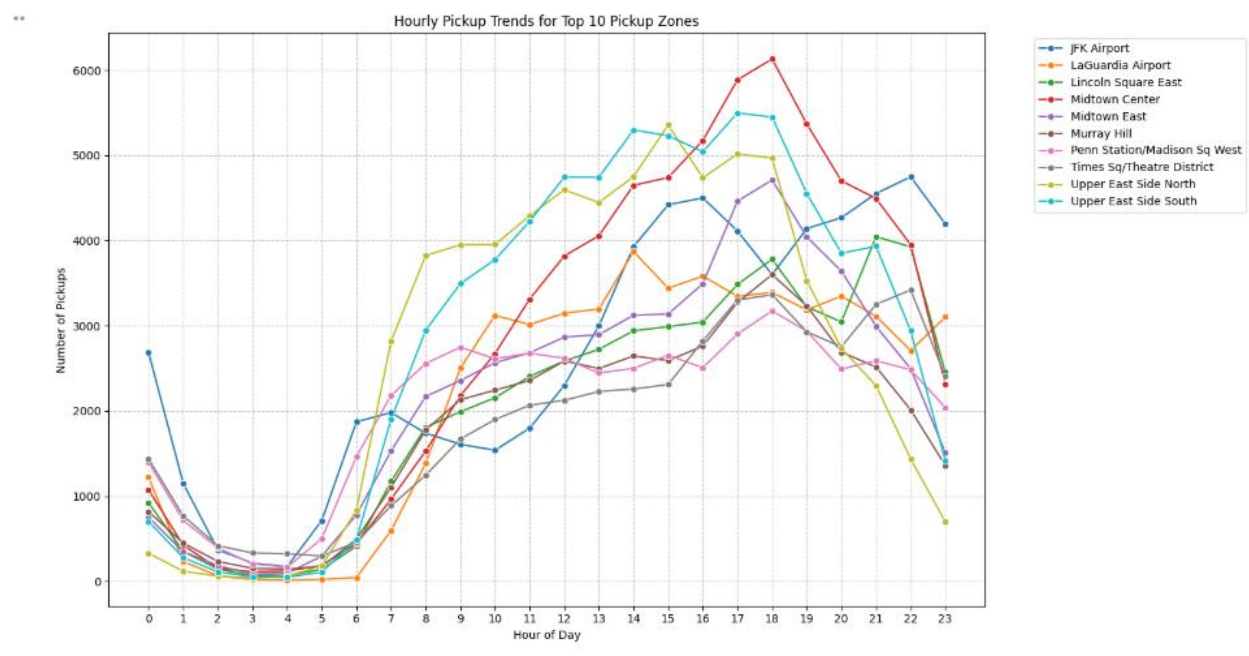
### 3.2.4. Compare hourly traffic on weekdays and weekends



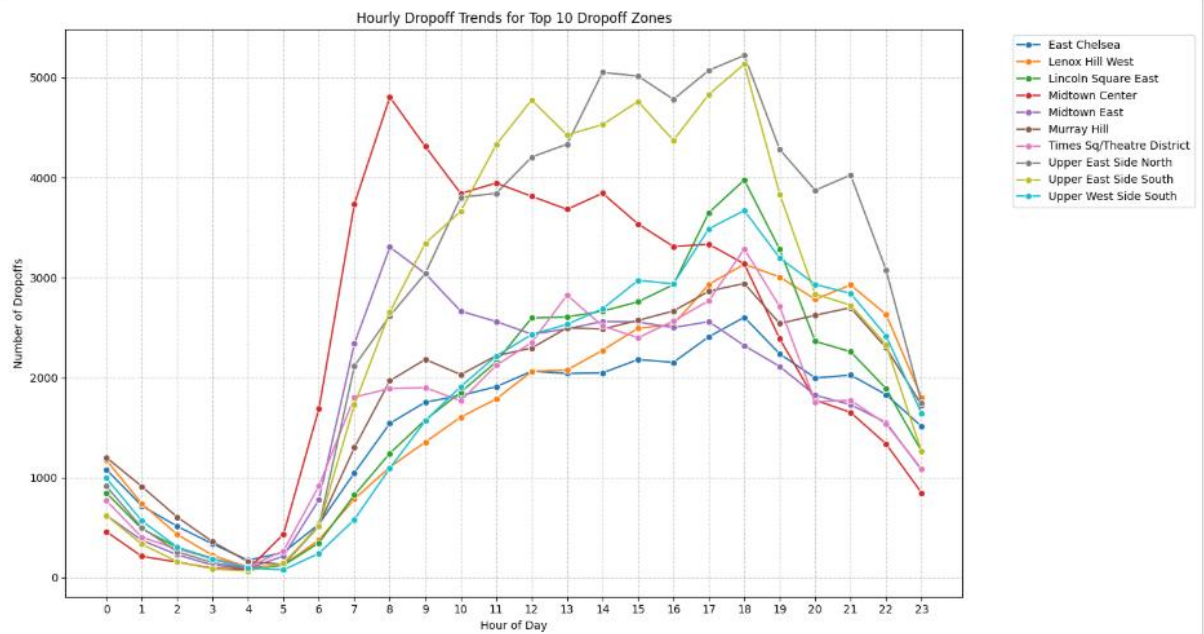
Comparatively, we have less number of pickups during weekends and more on weekdays. During midnight, demand is slightly more on weekend than weekdays.

### 3.2.5. Identify the top 10 zones with high hourly pickups and drops

Hourly Pickup Trend for top 10 zones



Hourly Dropoff Trend for top 10 zones



### 3.2.6. Find the ratio of pickups and dropoffs in each zone



Top 10 Zones with Highest Pickup/Dropoff Ratio:

|     | zone                             | total_pickups | total_dropoffs | pickup_dropoff_ratio |  |
|-----|----------------------------------|---------------|----------------|----------------------|--|
| 70  | East Elmhurst                    | 6428.0        | 473.0          | 13.589852            |  |
| 127 | JFK Airport                      | 63598.0       | 14918.0        | 4.263172             |  |
| 133 | LaGuardia Airport                | 51661.0       | 20011.0        | 2.581630             |  |
| 194 | Rikers Island                    | 2.0           | 0.0            | 2.000000             |  |
| 1   | Jamaica Bay                      | 2.0           | 0.0            | 2.000000             |  |
| 202 | Saint Michaels Cemetery/Woodside | 12.0          | 7.0            | 1.714286             |  |
| 181 | Penn Station/Madison Sq West     | 48951.0       | 29907.0        | 1.636774             |  |
| 109 | Greenwich Village South          | 20277.0       | 14617.0        | 1.387220             |  |
| 42  | Central Park                     | 24227.0       | 17494.0        | 1.384875             |  |
| 244 | West Village                     | 34551.0       | 25846.0        | 1.336803             |  |

Top 10 Zones with Lowest Pickup/Dropoff Ratio (excluding zero pickups):

|     | zone                     | total_pickups | total_dropoffs | pickup_dropoff_ratio |  |
|-----|--------------------------|---------------|----------------|----------------------|--|
| 144 | Madison                  | 1.0           | 122.0          | 0.008197             |  |
| 0   | Newark Airport           | 37.0          | 4248.0         | 0.008710             |  |
| 37  | Cambria Heights          | 1.0           | 107.0          | 0.009346             |  |
| 14  | Bay Terrace/Fort Totten  | 1.0           | 87.0           | 0.011494             |  |
| 18  | Bellerose                | 1.0           | 78.0           | 0.012821             |  |
| 160 | Midwood                  | 3.0           | 233.0          | 0.012876             |  |
| 254 | Woodlawn/Wakefield       | 1.0           | 75.0           | 0.013333             |  |
| 17  | Bedford Park             | 1.0           | 67.0           | 0.014925             |  |
| 166 | Murray Hill-Queens       | 2.0           | 128.0          | 0.015625             |  |
| 249 | Williamsbridge/Olinville | 1.0           | 64.0           | 0.015625             |  |

### 3.2.7. Identify the top zones with high traffic during night hours

Top 10 zones with highest pickup traffic during night hours (11 PM to 5 AM):

|     | zone                         | night_pickup_count |
|-----|------------------------------|--------------------|
| 52  | East Village                 | 13173              |
| 178 | West Village                 | 10732              |
| 85  | JFK Airport                  | 9500               |
| 99  | Lower East Side              | 8243               |
| 29  | Clinton East                 | 8201               |
| 72  | Greenwich Village South      | 7440               |
| 163 | Times Sq/Theatre District    | 5985               |
| 127 | Penn Station/Madison Sq West | 5415               |
| 113 | Midtown South                | 4905               |
| 42  | East Chelsea                 | 4806               |

Top 10 zones with highest dropoff traffic during night hours (11 PM to 5 AM):

|     | zone                  | night_dropoff_count |
|-----|-----------------------|---------------------|
| 78  | East Village          | 6864                |
| 46  | Clinton East          | 5217                |
| 164 | Murray Hill           | 5095                |
| 102 | Gramercy              | 4883                |
| 135 | Lenox Hill West       | 4601                |
| 67  | East Chelsea          | 4587                |
| 255 | Yorkville West        | 4371                |
| 241 | West Village          | 4177                |
| 231 | Upper West Side South | 3869                |
| 228 | Upper East Side North | 3786                |

### 3.2.8. Find the revenue share for nighttime and daytime hours

---

Nighttime Revenue Share: 12.05%  
Daytime Revenue Share: 87.95%

---

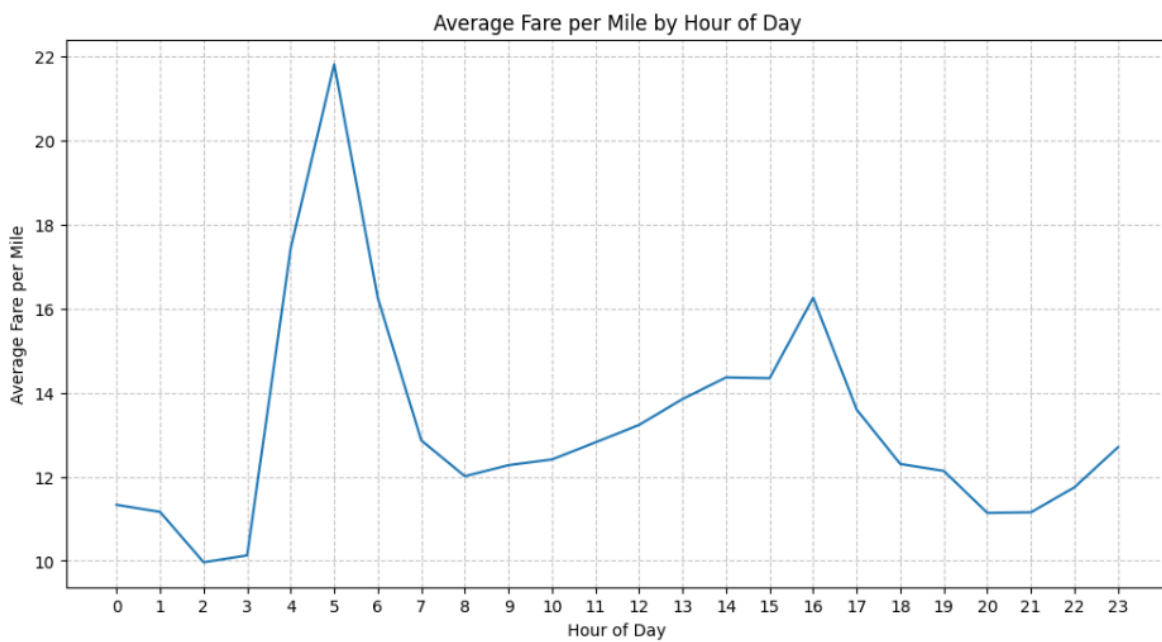
Revenue share is more during daytime and less on night time since the total trips are less.

### 3.2.9. For the different passenger counts, find the average fare per mile per passenger

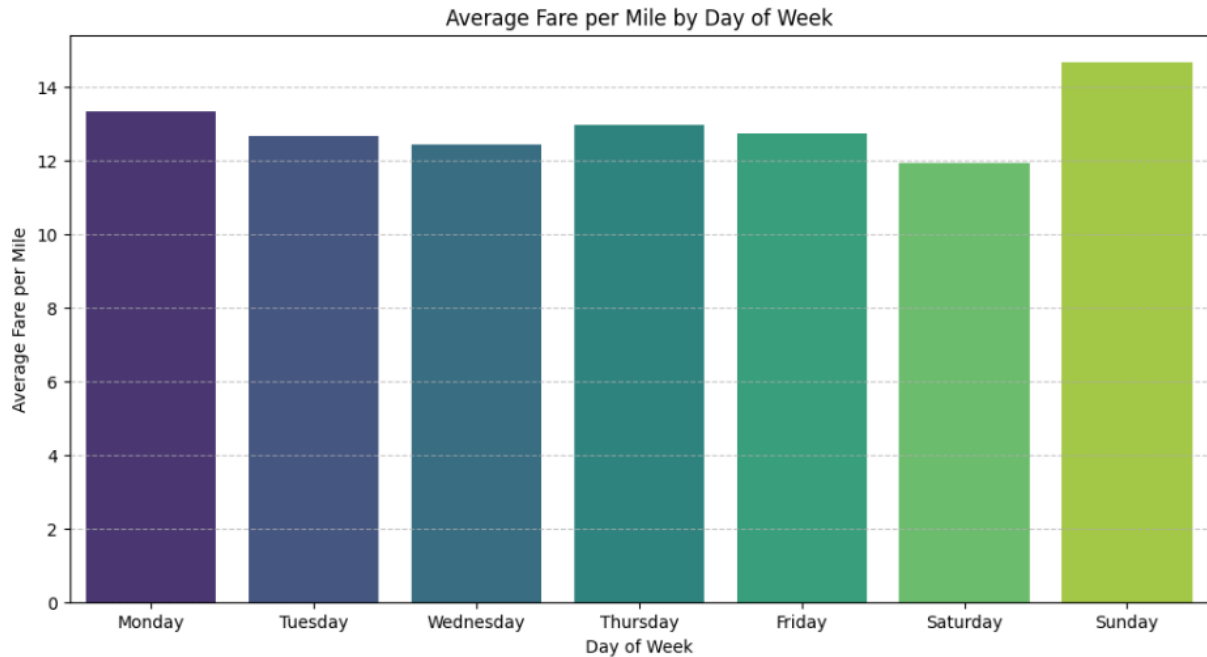
|   | passenger_count | fare_per_mile_per_passenger |
|---|-----------------|-----------------------------|
| 0 | 1.0             | 12.415541                   |
| 1 | 2.0             | 7.324027                    |
| 2 | 3.0             | 4.743307                    |
| 3 | 4.0             | 5.096074                    |
| 4 | 5.0             | 2.191935                    |
| 5 | 6.0             | 1.816143                    |

If the passenger count increases, the fare per mile per passenger decreases. Here, we have strong negative correlation.

### 3.2.10. Find the average fare per mile by hours of the day and by days of the week



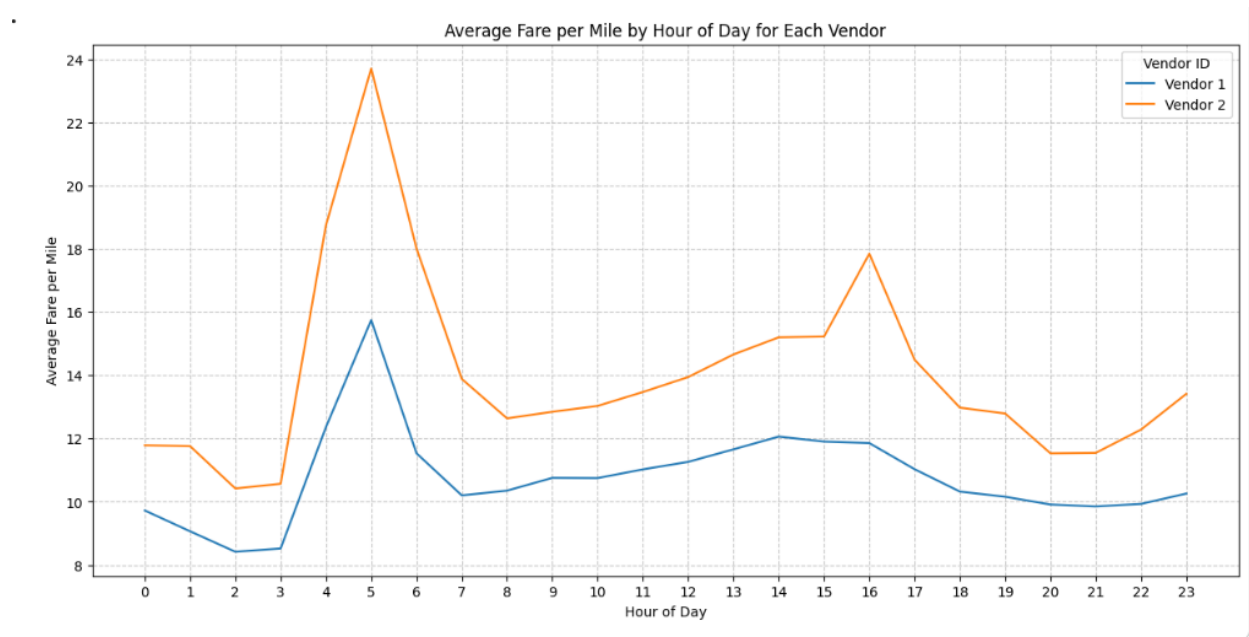
The fare is more at 5AM.



On Sunday, the fare is relatively higher than on other days.

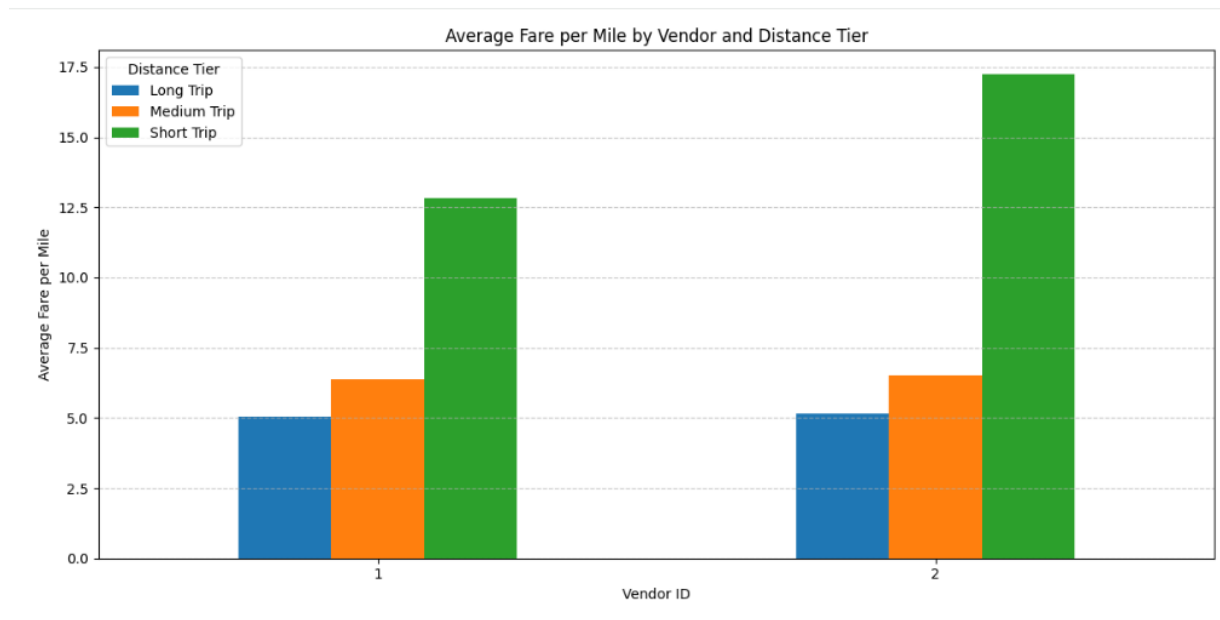
### 3.2.11. Analyse the average fare per mile for the different vendors

In other filtered dataset, we have only 2 vendors (1,2).



Vendor 2 is charging more than vendor 1.

### 3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

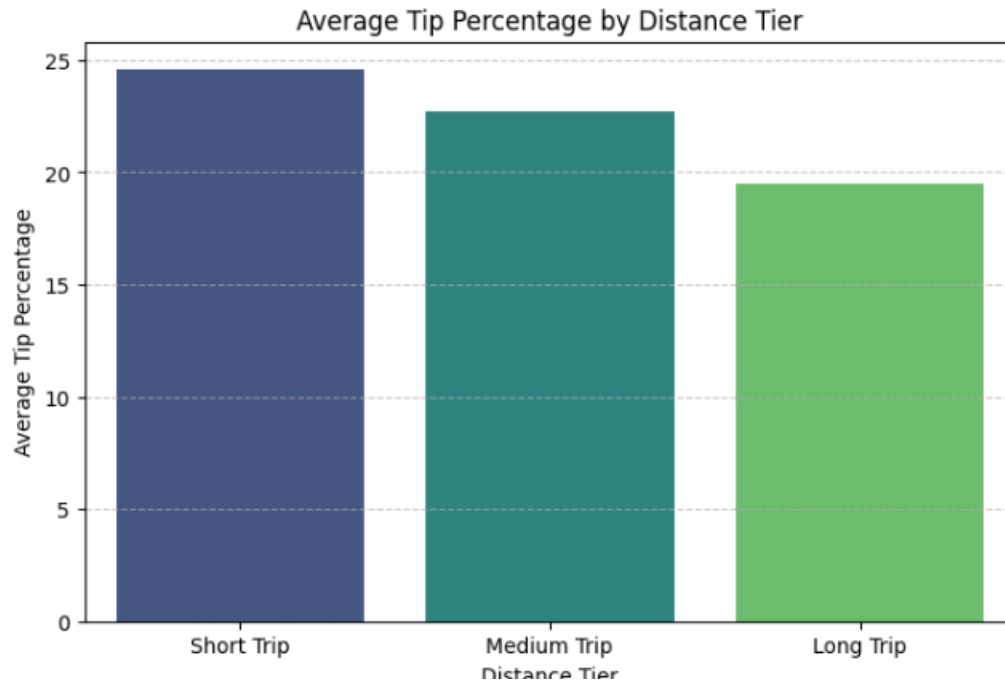


Both the vendors follow the same strategy on fare amount for different distance tiers.

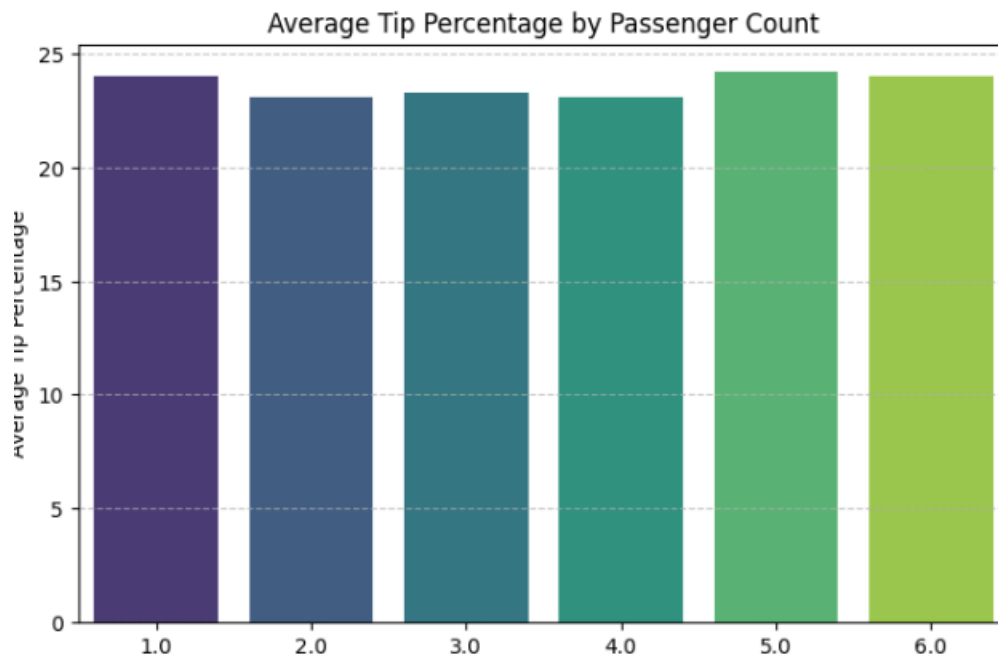
For short trip, both vendors are charging higher amount.

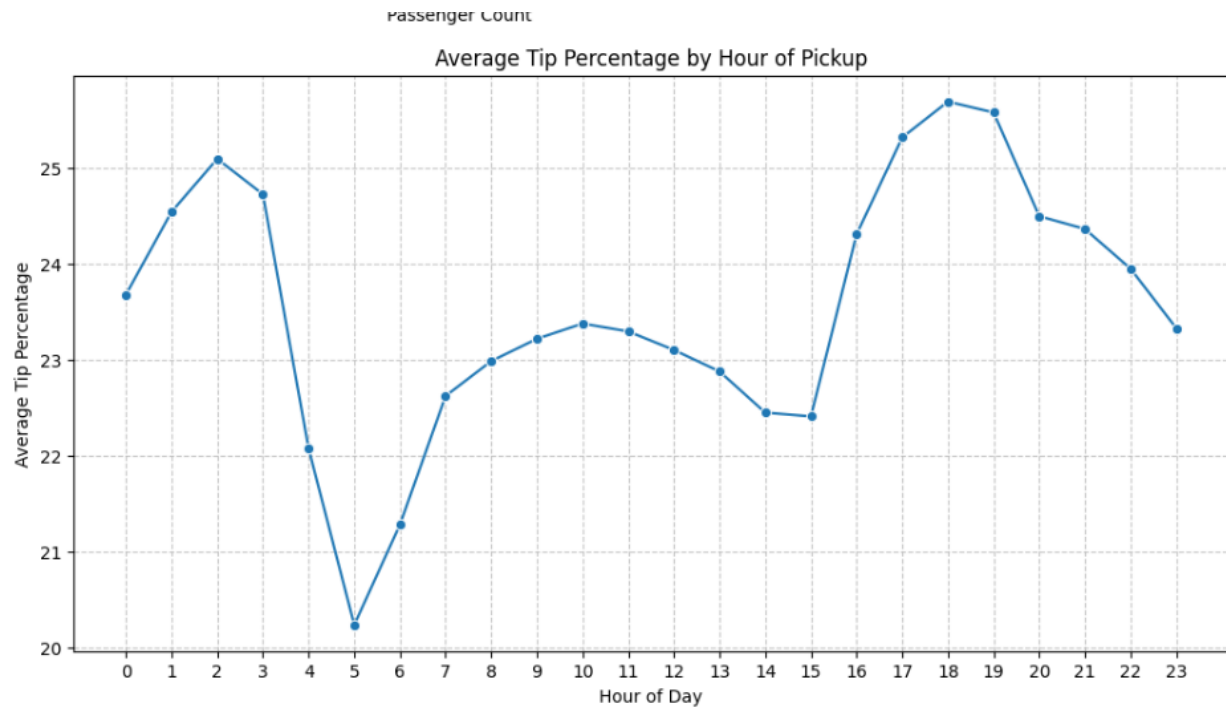
### 3.2.13. Analyse the tip percentages

```
sns.barplot(x=avg_tip_by_distance_tier.index, y=avg_tip_by_distance_tier.values, palette= viridis ,
```

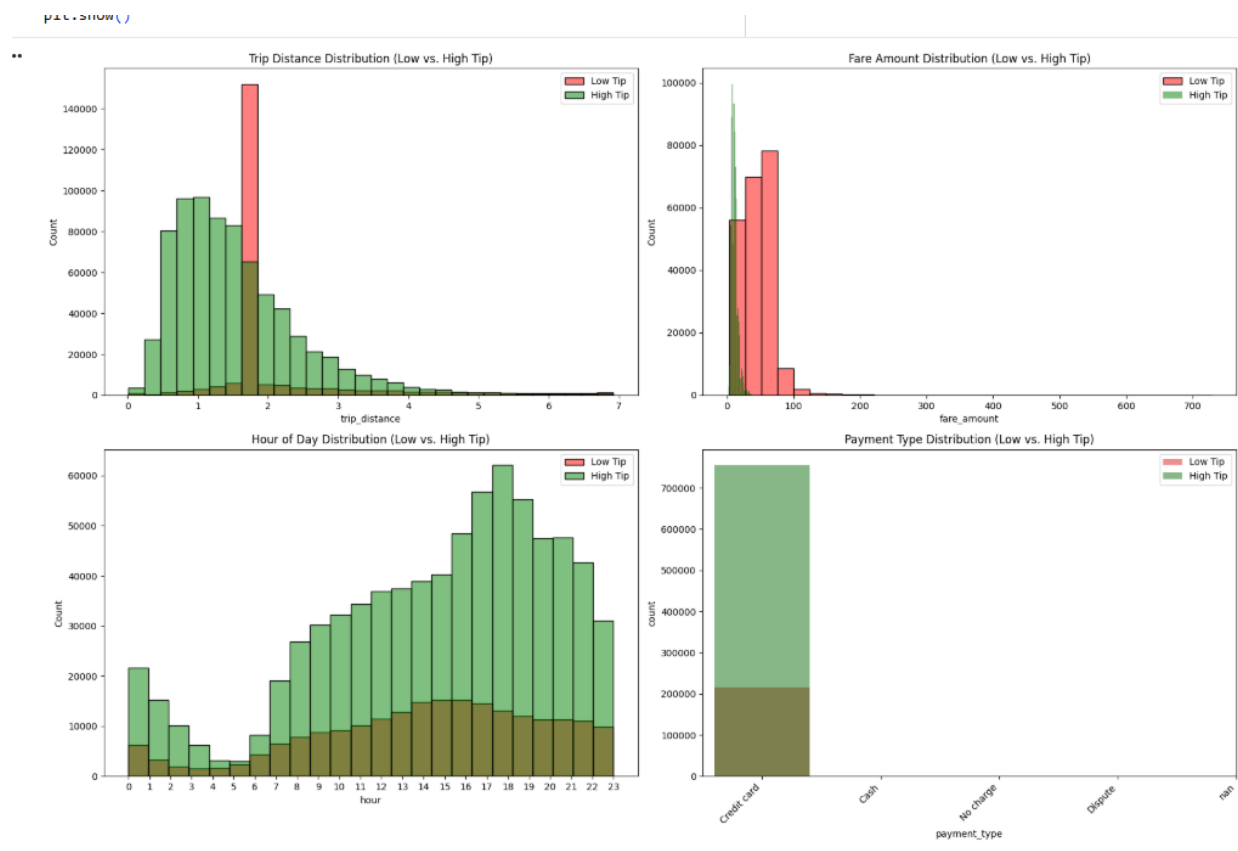


Average tip amount is more for short trip and less for long trip.

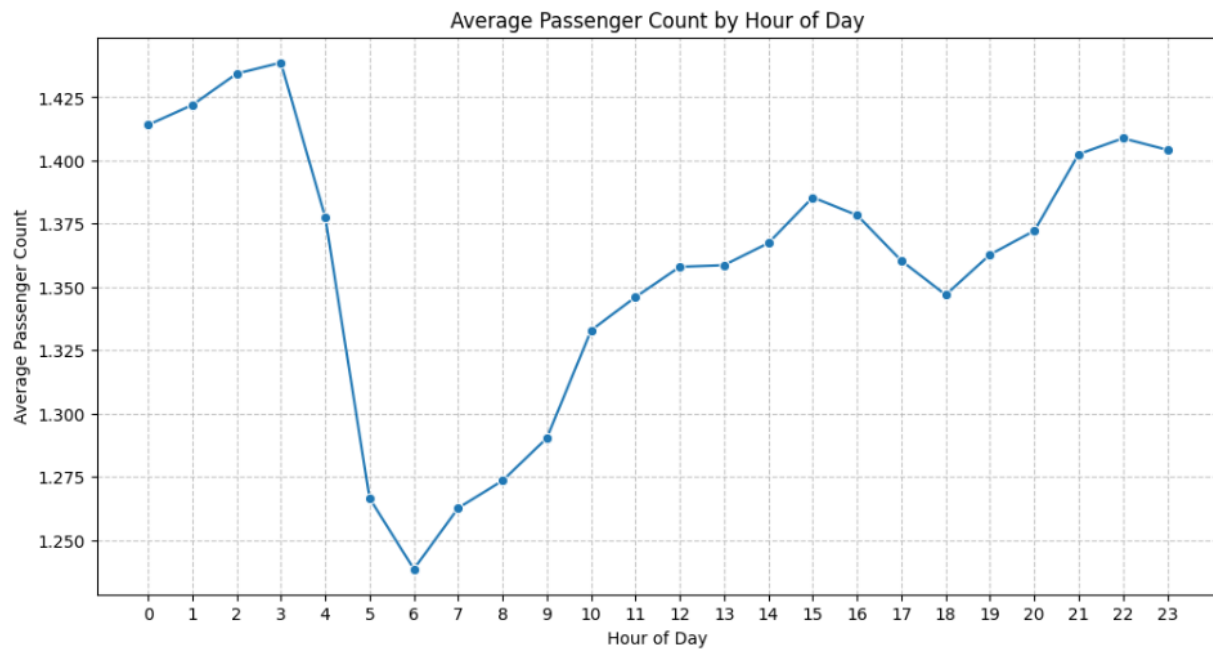




Tip percentage is more during peak hours.

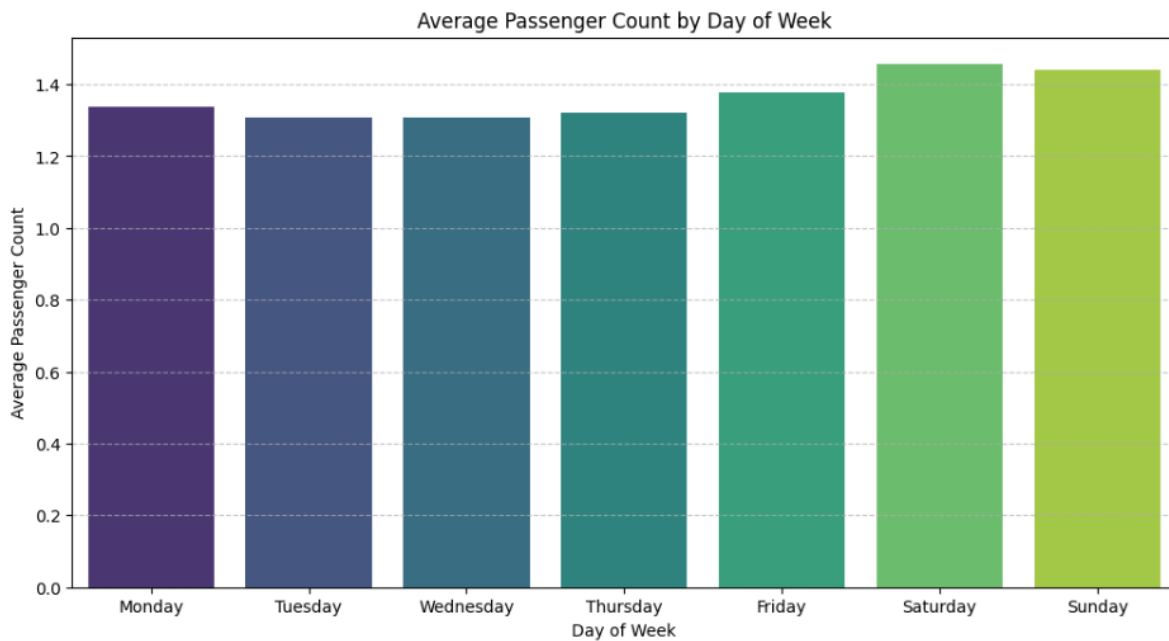


### 3.2.14. Analyse the trends in passenger count



/tmp/ipykernel\_9655/1291062171.py:11: FutureWarning:

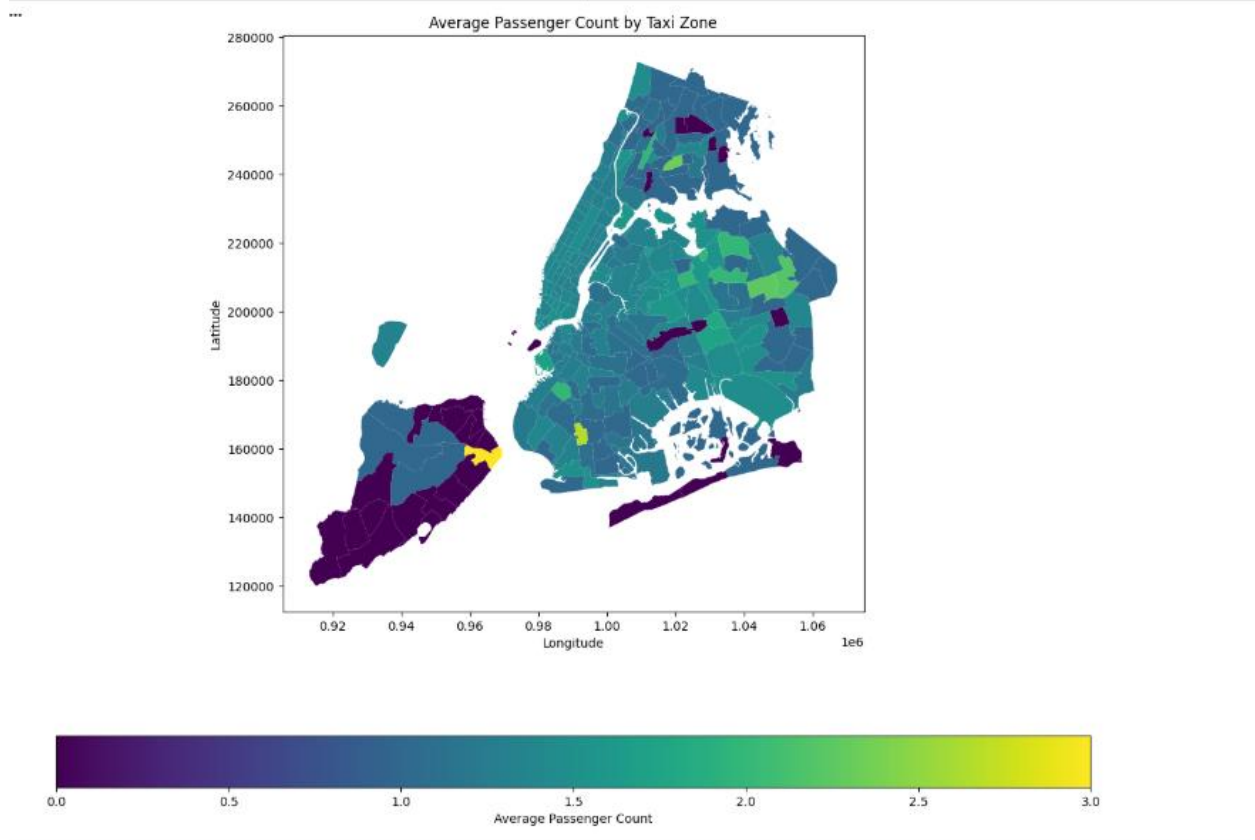
Average Passenger count during night hours is high



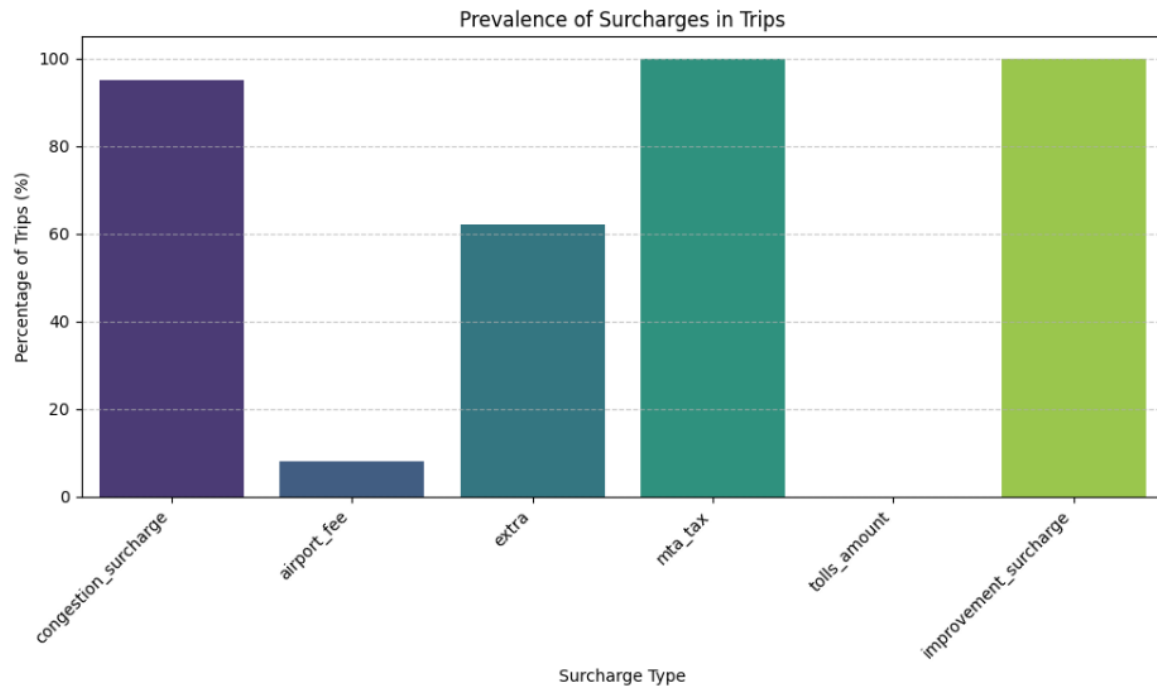
Average passenger count is slightly higher during weekends.



### 3.2.15. Analyse the variation of passenger counts across zones



### 3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.



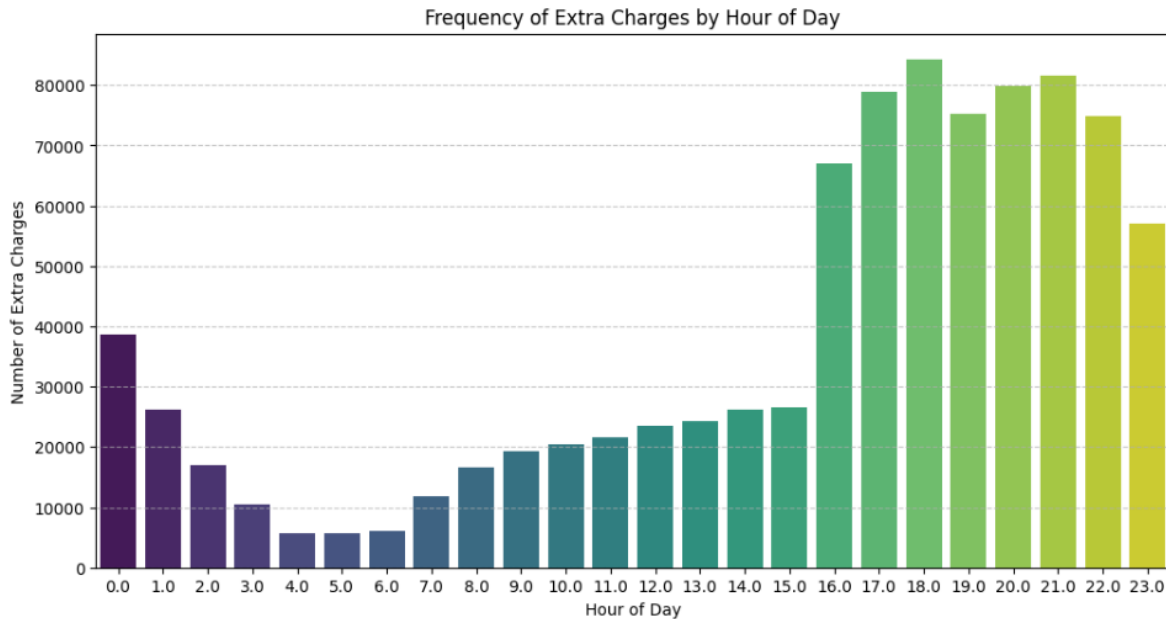
Mta\_tax and improvement\_surcharge are applicable for almost all trips. Tolls amount is added to very few trips.

.. Hourly distribution of 'extra' charges:

| hour | extra_charge_count |
|------|--------------------|
| 18   | 84318              |
| 21   | 81610              |
| 20   | 79943              |
| 17   | 78924              |
| 19   | 75186              |
| 22   | 74788              |
| 16   | 67069              |
| 23   | 57027              |
| 0    | 38656              |
| 15   | 26618              |
| 14   | 26201              |
| 1    | 26140              |
| 13   | 24299              |
| 12   | 23434              |

During peak hours, extra amount has been added more frequently.

```
sns.barplot(x='hour', y='extra_charge_count', data=extra_by_hour, palette='viridis')
```



Early Mornings, extra amounts are less frequently applying.

## 4. Conclusions

### 4.1. Final Insights and Recommendations

#### 4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies

1. High demand hours and days:

- **Peak Hours (17:00-19:00 daily):** Our analysis shows that 5 PM to 7 PM are the busiest hours, and at 6 PM being the absolute peak. This consistent pattern across weekdays suggests a need for increased driver availability and strategic vehicle deployment during these times. During these hours, areas with high pickup and dropoff activity like Midtown Center, Upper East Side South and North, and JFK Airport should have a higher number of available taxis.
- **Weekend Operations:** Weekends, particularly Saturdays, exhibit high demand. Staffing levels should be adjusted to meet this increased demand, especially in popular evening and night-time zones.

2. Route Optimization and avoid traffic:

- **Slowest Routes Identification:** The identification of slowest routes (slowest\_routes) should be a continuous process. Dispatch systems can use this information to actively

suggest alternative routes to drivers during specific hours, especially for trips originating or terminating in these bottleneck areas.

- **Real-time Traffic Integration:** Implement or enhance systems that integrate real-time traffic data to dynamically adjust suggested routes, minimizing delays and improving efficiency.

3. Strategic Positioning based on Zone Activity:

- **High Pickup/Dropoff Zones:** Continuously monitor and predict demand in top pickup and dropoff zones (e.g., Upper East Side South, Midtown Center, JFK Airport). Pre-positioning vehicles in these high-demand areas before peak times can significantly reduce passenger wait times and increase driver utilization.
- **Night-time Hotspots:** Focus on deploying more taxis in top night-time pickup zones (East Village, JFK Airport, Times Sq/Theatre District) and dropoff zones (East Village, Clinton East, Murray Hill) during 11 PM to 5 AM. This will cater to late-night demand and capitalize on potential revenue.

4. Balancing Pickup/Dropoff Ratios:

- **High Pickup/Low Dropoff Ratio Zones:** For zones with a high pickup-to-dropoff ratio (e.g., LGA Airport, JFK Airport, The Rockaways), implement strategies to encourage drivers to move towards these areas, perhaps through incentives or directed dispatch, after dropping off passengers in low-pickup zones.
- **Low Pickup/High Dropoff Ratio Zones:** Conversely, for zones with low pickup-to-dropoff ratios (e.g., Newark Airport, Midwood, Bedford Park), consider having fewer taxis waiting after drop-offs and guide them to more active zones.

5. Leveraging Passenger Count Data:

- **Predictive Dispatch:** While average passenger count doesn't show extreme variations, using historical data on passenger counts per hour and day can help predict the type of vehicle needed (e.g., larger vehicles for potential group rides) in certain areas, improving service matching.

---

#### 4.1.2 [5 marks]

Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

---

#### Strategic Positioning of Cabs based on Temporal and Geographical Trends

To optimize cab positioning, a dynamic strategy that adapts to hourly, daily, and monthly demand patterns, as well as specific zone characteristics, is crucial.

1. Hourly Pre-positioning in High-Demand Zones:

- **Morning Rush (7 AM - 9 AM):** Deploy more cabs to residential areas in boroughs like Queens and Brooklyn (e.g., Astoria, Long Island City) and zones surrounding major transit hubs (e.g., Grand Central, Penn Station) in Manhattan, anticipating commuter demand.
- **Lunchtime Peak (12 PM - 2 PM):** Increase cab presence around commercial districts (Midtown, Financial District) and tourist attractions, as well as areas with high office concentration for midday travel.

- **Evening Rush (5 PM - 7 PM):** This is the busiest period. Focus deployment heavily on Midtown Center, Upper East Side South/North, and JFK Airport. Ensure a strong presence in business districts for homeward-bound commuters and entertainment areas for evening activities.
- **Late Night/Early Morning (11 PM - 5 AM):** Shift focus to night-time hotspots identified (East Village, Times Square/Theatre District) for pickups, and residential areas in boroughs like Queens, Brooklyn, and Staten Island for drop-offs. Airports (JFK, LGA) also maintain significant demand during these hours for arrivals and departures.

## 2. Day-of-Week Specific Strategies:

- **Weekdays (Monday-Friday):** Maintain higher density in business and commercial districts during working hours. Evenings should see a shift towards entertainment and dining areas.
- **Weekends (Saturday-Sunday):** Anticipate higher demand in recreational areas, tourist spots, shopping districts, and airports. Saturday evenings are particularly busy, requiring maximum driver availability and strategic positioning in entertainment hubs across Manhattan and parts of Brooklyn.

## 3. Monthly/Seasonal Adjustments:

- **Peak Months (May, October, November):** During these months, overall demand is higher. This necessitates a general increase in fleet availability and more aggressive pre-positioning strategies across all high-demand zones.
- **Seasonal Events:** Proactively adjust cab distribution for major events (e.g., Fashion Week, holiday seasons, sports events) in specific zones. This might involve setting up temporary staging areas or incentivizing drivers to operate in these locations.

## 4. Balancing Pickup and Dropoff Ratios (Geographical Flows):

- **High Pickup Ratio Zones (e.g., Airports, The Rockaways):** Encourage drivers to move towards these zones after completing trips in areas with low pickup ratios, potentially through dynamic surge pricing for entering these areas or bonus incentives.
- **Low Pickup Ratio Zones (e.g., Newark Airport, Midwood, Bedford Park):** After dropping off passengers in these areas, guide drivers to nearby higher-demand zones or incentivize them to take on trips back towards the city center.

## 5. Leveraging Real-time Data and Predictive Analytics:

- **Dynamic Re-routing:** Implement a system that can quickly identify emerging high-demand pockets (e.g., after a concert, sporting event, or during unexpected weather changes) and dynamically redirect available cabs.
- **Driver Incentives:** Use real-time data to offer targeted incentives to drivers to position themselves in areas or during times where supply is low relative to demand, particularly in slow routes during specific hours to improve efficiency.

### 4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

## Strategic Positioning of Cabs based on Temporal and Geographical Trends

To optimize cab positioning, a dynamic strategy that adapts to hourly, daily, and monthly demand patterns, as well as specific zone characteristics, is crucial.

### **1. Hourly Pre-positioning in High-Demand Zones:**

- Morning Rush (7 AM - 9 AM): Deploy more cabs to residential areas in boroughs like Queens and Brooklyn (e.g., Astoria, Long Island City) and zones surrounding major transit hubs (e.g., Grand Central, Penn Station) in Manhattan, anticipating commuter demand.
- Lunchtime Peak (12 PM - 2 PM): Increase cab presence around commercial districts (Midtown, Financial District) and tourist attractions, as well as areas with high office concentration for midday travel.
- Evening Rush (5 PM - 7 PM): This is the busiest period. Focus deployment heavily on Midtown Center, Upper East Side South/North, and JFK Airport. Ensure a strong presence in business districts for homeward-bound commuters and entertainment areas for evening activities.
- Late Night/Early Morning (11 PM - 5 AM): Shift focus to night-time hotspots identified (East Village, Times Square/Theatre District) for pickups, and residential areas in boroughs like Queens, Brooklyn, and Staten Island for drop-offs. Airports (JFK, LGA) also maintain significant demand during these hours for arrivals and departures.

### **2. Day-of-Week Specific Strategies:**

- Weekdays (Monday-Friday): Maintain higher density in business and commercial districts during working hours. Evenings should see a shift towards entertainment and dining areas.
- Weekends (Saturday-Sunday): Anticipate higher demand in recreational areas, tourist spots, shopping districts, and airports. Saturday evenings are particularly busy, requiring maximum driver availability and strategic positioning in entertainment hubs across Manhattan and parts of Brooklyn.

### **3. Monthly/Seasonal Adjustments:**

- Peak Months (May, October, November): During these months, overall demand is higher. This necessitates a general increase in fleet availability and more aggressive pre-positioning strategies across all high-demand zones.
- Seasonal Events: Proactively adjust cab distribution for major events (e.g., Fashion Week, holiday seasons, sports events) in specific zones. This might involve setting up temporary staging areas or incentivizing drivers to operate in these locations.

### **4. Balancing Pickup and Dropoff Ratios (Geographical Flows):**

- High Pickup Ratio Zones (e.g., Airports, The Rockaways): Encourage drivers to move towards these zones after completing trips in areas with low pickup ratios, potentially through dynamic surge pricing for entering these areas or bonus incentives.
- Low Pickup Ratio Zones (e.g., Newark Airport, Midwood, Bedford Park): After dropping off passengers in these areas, guide drivers to nearby higher-demand zones or incentivize them to take on trips back towards the city center.

### **5. Leveraging Real-time Data and Predictive Analytics:**

- Dynamic Re-routing: Implement a system that can quickly identify emerging high-demand pockets (e.g., after a concert, sporting event, or during unexpected weather changes) and dynamically redirect available cabs.

- **Driver Incentives:** Use real-time data to offer targeted incentives to drivers to position themselves in areas or during times where supply is low relative to demand, particularly in slow routes during specific hours to improve efficiency.

#### 4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

##### 1. Dynamic Surge Pricing based on Time and Location:

- **Hourly Peaks:** Implement surge pricing during peak hours (e.g., 5 PM - 7 PM, especially around 6 PM) when demand significantly outstrips supply, as identified by the hourly pickup trends. This is particularly effective in high-demand zones like Midtown Center, Upper East Side, and JFK Airport.
- **Weekend Pricing:** Apply higher multipliers during weekends, especially Saturday evenings and nights, given the consistently high demand during these periods.
- **Targeted Zone-Based Surges:** Use real-time demand data to apply micro-surges in specific high-pickup zones during their respective peak times, rather than a blanket surge across the city. This can also be used to incentivize drivers to move into these areas.

##### 2. Tiered Pricing based on Distance:

- **Optimize Fare Per Mile by Tier:** The analysis of 'average fare per mile by distance tier' shows variations. Consider adjusting the base fare and per-mile rates for 'Short Trips' ( $\leq 2$  miles), 'Medium Trips' (2-5 miles), and 'Long Trips' ( $> 5$  miles) to reflect customer willingness to pay and operational costs for different trip lengths. For example, if short trips have a disproportionately high fare/mile, it might indicate room for slight adjustments to capture more volume without significantly impacting revenue, or conversely, to maintain higher profitability on short, quick trips.
- **Review Minimum Fares:** For very short distances, ensure that minimum fares adequately cover operational costs and provide sufficient incentive for drivers.

##### 3. Competitive Adjustments based on Vendor Performance:

- **Vendor-Specific Analysis:** Regularly compare average fare per mile and overall pricing structures with competitors (Vendor 1 vs. Vendor 2). If one vendor consistently charges less for certain tiers or hours without a clear operational advantage, evaluate if prices can be adjusted to be more competitive or to capitalize on perceived value.
- **Value Proposition:** If the company can offer superior service (e.g., faster pickups, better vehicle quality), it may justify a slightly higher premium in certain segments, provided this is communicated effectively to customers.

##### 4. Surcharge Optimization and Transparency:

- **Review Surcharge Application:** Analyze the prevalence of surcharges (congestion, airport fee, extra). Ensure that surcharges are applied consistently and transparently. If a surcharge is rarely applied, evaluate its necessity or the conditions under which it should be triggered.

- **Dynamic Surcharge for Specific Conditions:** For 'extra' charges, which are frequently applied during certain hours (e.g., evening rush), consider if these can be integrated more smoothly into a dynamic pricing model rather than being seen as arbitrary add-ons.

5. Leveraging Passenger Count for Pricing:

- **Group Ride Discounts/Premiums:** While average passenger\_count doesn't strongly correlate with fare, exploring group ride options with dynamic pricing could cater to different market segments. Alternatively, if multiple passengers imply higher wear-and-tear or longer boarding times, a small premium could be considered, though this needs careful evaluation against customer experience.

6. Tip-Influenced Strategies:

- **Service Quality Incentives:** Since tip percentages vary by distance tier and hour, focus on enhancing service quality during periods or for trip types that historically yield lower tips. This could indirectly boost driver earnings and overall customer satisfaction.
- **Driver Education:** Educate drivers on factors that influence tipping behavior (e.g., smooth rides, professional demeanor) to encourage better service, especially during times/trips where tips are typically lower.

By systematically implementing these data-driven pricing adjustments, the taxi operation can dynamically respond to market conditions, optimize revenue streams, and maintain a strong competitive position.